

KHÓA ĐÀO TẠO · 8 GIỜ · 5 SESSION

Jenkin & Tự động hóa CI/CD

DevOps Automation · Jenkins · Ansible · Config as Code · Golang

Tự động hóa quy trình Build → Deploy → Quản lý cấu hình, và tự viết công cụ DevOps bằng Go. Tài liệu chú trọng **vì sao (why)** đằng sau mỗi công cụ.

Mục lục

PHẦN I — NỘI DUNG

1.	Session 1 — CI/CD & Jenkins cơ bản	4
2.	Session 2 — Jenkins Pipeline & tự động hóa Build/Deploy	12
3.	Session 3 — Ansible: Configuration Management	22
4.	Session 4 — Configuration as Code: hệ sinh thái công cụ	31
5.	Session 5 — Golang để viết DevOps tool	40

PHẦN II — BÀI GIẢI LAB

1.	Bài giải Lab 1 — Dựng Jenkins & Freestyle job	51
2.	Bài giải Lab 2 — Jenkinsfile Build → Test → Deploy	58
3.	Bài giải Lab 3 — Ansible playbook cài Nginx & tích hợp Jenkins	65
4.	Bài giải Lab 4 — Terraform & Docker Compose (Config as Code)	73
5.	Bài giải Lab 5 — CLI tool DevOps bằng Go	81

PHẦN III — THAM KHẢO

1.	Reference — Ebook & Video	90
----	---------------------------	----

PHẦN I

Nội dung bài giảng

Session 1

1h30

CI/CD

Jenkins

Session 1 — CI/CD & Jenkins cơ bản

Mục tiêu: Hiểu bản chất CI/CD và dựng được Jenkins kết nối với Git repo.

Hãy tưởng tượng một nhóm 5 lập trình viên. Mỗi người ôm một nhánh code riêng suốt hai tuần, rồi đến ngày "gộp chung". Kết quả: hàng trăm dòng xung đột, code chạy được trên máy A nhưng vỡ trên máy B, không ai dám bấm deploy vì "chưa biết có gây gì không". Cảnh này có tên riêng: **integration hell**. CI/CD sinh ra để dập tắt nó. Session này không dạy bạn "bấm nút nào", mà giải thích **vì sao** những nút đó tồn tại.

Bức tranh lớn: vì sao cần CI/CD

Phần mềm chỉ tạo ra giá trị khi nó **chạy trên tay người dùng**, không phải khi nó nằm trên máy lập trình viên. Khoảng cách giữa "code vừa viết xong" và "người dùng chạm được" chính là nơi mọi rủi ro tích tụ: merge thủ công, build thủ công, test thủ công, copy file lên server thủ công. Mỗi bước "thủ công" là một điểm để con người quên, làm sai, hoặc làm khác nhau giữa các lần.

CI/CD là triết lý **tự động hóa toàn bộ con đường đó** và **rút ngắn vòng phản hồi (feedback loop)**. Ý tưởng cốt lõi: nếu tích hợp code là việc đau đớn, thì hãy làm nó **thường xuyên đến mức nó không còn đau nữa**. Một vết thương lớn hai tuần một lần đáng sợ hơn nhiều so với một vết xước nhỏ mỗi giờ.

KHÔNG CÓ CI/CD

code 2 tuần
code 2 tuần
code 2 tuần

gộp 1 lần → BÙM
(integration hell)

CÓ CI/CD

push nhỏ
push nhỏ
push nhỏ

tự build+test mỗi lần
lỗi lộ ra trong vài phút

Insight: CI/CD không phải một công cụ, mà là một **chiến lược quản lý rủi ro**. Công cụ (Jenkins) chỉ là cách hiện thực hóa chiến lược đó.

Nếu **không** có CI/CD, nỗi đau nằm ở đâu? Lỗi được phát hiện muộn (deploy xong khách hàng mới báo), chi phí sửa tăng theo cấp số nhân theo thời gian, và tâm lý “sợ release” khiến team gom nhiều thay đổi vào một lần deploy khổng lồ — càng gom nhiều, càng dễ gãy, càng sợ, tạo thành vòng xoáy.

CI, CD và CD: ba khái niệm dễ nhầm

Chữ “CD” bị dùng cho hai khái niệm khác nhau, nên rất dễ loạn. Hãy tách bạch theo **mức độ tự động và ranh giới trách nhiệm**.

Continuous Integration (CI) — Tích hợp liên tục. Mỗi lần lập trình viên push code, hệ thống **tự động build và chạy test**. Mục tiêu duy nhất: trả lời câu hỏi “code mới có tích hợp được với code cũ mà không gây không?” trong vài phút. CI dừng lại ở chỗ tạo ra một bản build “đã được kiểm chứng”.

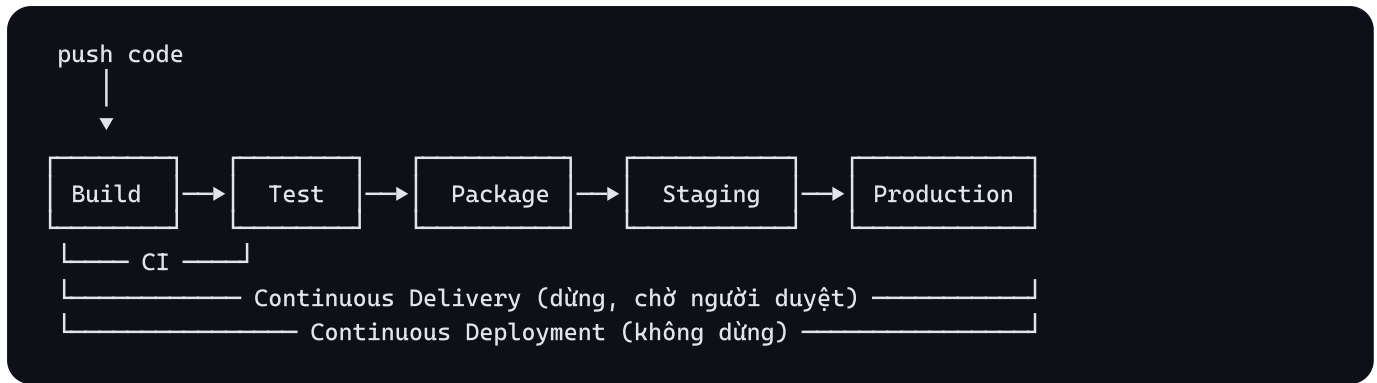
Continuous Delivery (CD) — Chuyển giao liên tục. Mở rộng CI: sau khi build+test xong, hệ thống tự động đóng gói và đưa lên môi trường staging, **luôn sẵn sàng để release**. Nhưng bước bấm nút lên production vẫn do **con người quyết định**.

Continuous Deployment (CD) — Triển khai liên tục. Đi xa nhất: nếu mọi test đều xanh, code **tự động lên thẳng production** mà không cần ai bấm nút.

Khía cạnh	Continuous Delivery	Continuous Deployment
Lên production	Con người bấm nút	Tự động hoàn toàn
Cần gì để an toàn	Test tốt + quy trình duyệt	Test rất tốt + rollback nhanh
Phù hợp với	Sản phẩm cần kiểm soát release	Sản phẩm cần tốc độ, web quy mô lớn
Rủi ro	Thấp hơn, chậm hơn	Cao hơn, nhanh hơn

Vì sao tách ra ba khái niệm? Vì mỗi mức đòi hỏi một **độ trưởng thành khác nhau về test và văn hóa**. Continuous Deployment mà thiếu test tự động là tự sát: bạn đang tự động đẩy lỗi lên đầu người dùng. Việc tách bạch giúp team biết mình đang ở đâu và cần bổ sung gì để tiến lên mức tiếp theo.

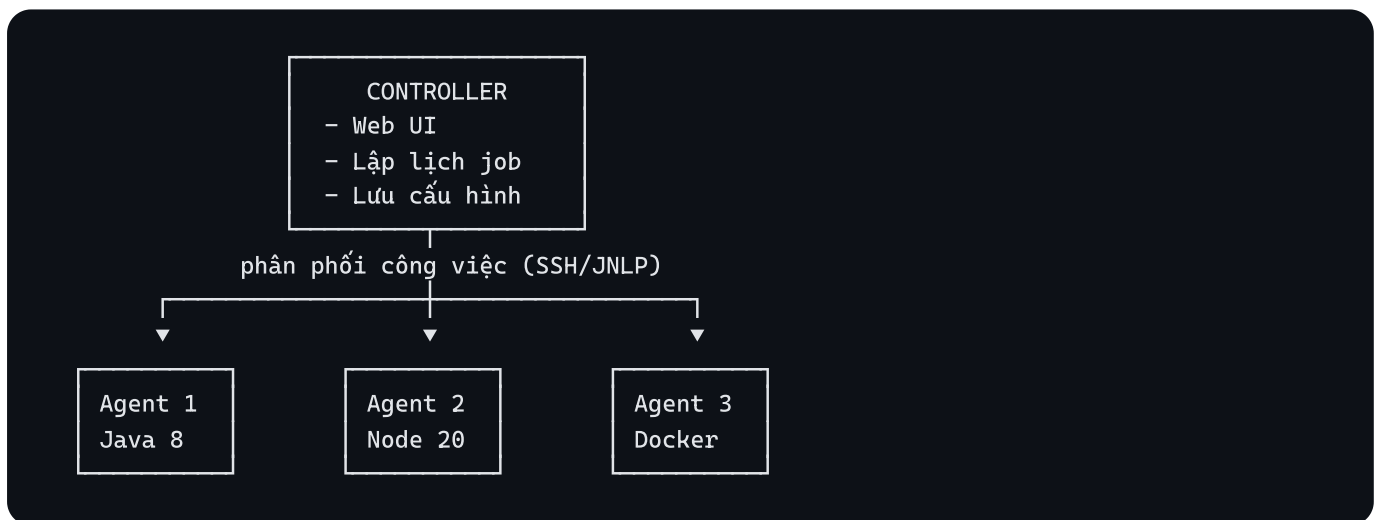
Luồng tổng thể có thể hình dung như một đường ống (pipeline):



Vì sao Jenkins có kiến trúc Controller/Agent + plugin

Jenkins là “nhạc trưởng” điều phối pipeline. Kiến trúc của nó phản ánh trực tiếp các bài toán thực tế mà một hệ thống CI phải giải.

Controller/Agent — vì sao tách đôi? Controller (trước đây gọi là master) là bộ não: giữ cấu hình, hiển thị giao diện web, lập lịch job, phân phối công việc. Nhưng nếu để chính controller **chạy build**, ta gặp ba vấn đề: (1) một build nặng có thể làm treo cả hệ thống; (2) mỗi project cần môi trường khác nhau — dự án A cần Java 8, dự án B cần Node 20 — không thể nhồi hết vào một máy; (3) không mở rộng được khi số lượng build tăng. Vì thế Jenkins tách **agent** (trước đây gọi là slave) ra: agent là các máy “làm việc tay chân”, nhận lệnh từ controller và thực thi build trong môi trường riêng của chúng.



Insight: Tách controller/agent chính là áp dụng nguyên tắc “**một việc, một chỗ**”. Controller lo điều phối, agent lo thực thi. Nhờ vậy bạn scale ngang bằng cách thêm agent, và cô lập môi trường để build của dự án này không đầu độc dự án khác.

Plugin — vì sao lỗi nhỏ + cấm thêm? Thế giới công cụ (Git, Docker, Slack, AWS, SonarQube...) thay đổi liên tục và nhiều vô kể. Nếu Jenkins tích hợp cứng tất cả vào lõi, lỗi sẽ phình to, khó bảo trì, và mỗi lần một công cụ đổi API là phải vá cả Jenkins. Giải pháp: **lỗi Jenkins cố tình nhỏ**, chỉ lo điều phối, còn mọi tích hợp cụ thể được đẩy ra **plugin** — hơn 1800 plugin do cộng đồng phát triển độc lập. Đây là kiến trúc “microkernel”: ổn định ở lõi, linh hoạt ở rìa. Cái giá phải trả là bạn phải tự chọn và quản lý plugin, nhưng đổi lại là sự linh hoạt gần như vô hạn.

Job vs Pipeline — và vì sao nên dùng Pipeline

Thời kỳ đầu, Jenkins cấu hình build qua **Freestyle job**: bạn click chuột qua giao diện web để chọn “lấy code từ đâu, chạy lệnh gì, gửi mail cho ai”. Đơn giản để bắt đầu, nhưng bộc lộ điểm yếu chết người khi quy trình phức tạp lên.

Vì sao **Pipeline** ra đời thay thế? Cốt lõi là triết lý “**Pipeline as Code**” — mô tả toàn bộ quy trình bằng một file text (**Jenkinsfile**) nằm **ngay trong Git repo** cùng với source code.

Tiêu chí	Freestyle job	Pipeline
Cấu hình	Click chuột trên UI	Viết code trong Jenkinsfile
Version control	Không (kẹt trong Jenkins)	Có (cùng repo với code)
Quy trình nhiều bước	Vụng về, khó nối	Tự nhiên (stages)
Chạy song song	Rất hạn chế	Hỗ trợ tốt
Xem lại lịch sử thay đổi	Khó	git log như code thường
Khôi phục khi hỏng	Khó, dễ mất cấu hình	Chỉ cần checkout lại repo

Một **Jenkinsfile** tối giản trông như sau:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'npm ci && npm run build' }
    }
    stage('Test') {
      steps { sh 'npm test' }
    }
  }
}
```

Giải thích: **agent any** bảo Jenkins chạy trên bất kỳ agent rảnh nào; mỗi **stage** là một giai đoạn hiển thị rõ ràng trên giao diện; **steps** là các lệnh shell thực thi. Toàn bộ quy trình giờ là một **artifact có thể review, diff, rollback** — hệt như code.

Vì sao điều này quan trọng? Cấu hình Freestyle sống trong bụng Jenkins: server hỏng là mất, không ai biết ai đổi gì và khi nào. Pipeline biến quy trình build thành **một phần của source code** — minh bạch, tái tạo được, và đi theo từng nhánh (branch A có thể có pipeline khác branch B).

Kết nối Git: credentials & webhook (và vì sao)

Jenkins cần lấy code từ Git repo. Có hai vấn đề nền tảng: **làm sao xác thực an toàn** và **làm sao biết khi nào có code mới**.

Credentials — vì sao không hardcode token? Tuyệt đối không viết username/password/token thẳng vào Jenkinsfile hay cấu hình job. Lý do: Jenkinsfile nằm trong Git, mà Git thì được clone khắp nơi và lưu lịch sử vĩnh viễn — hardcode token nghĩa là **rò rỉ bí mật cho bất kỳ ai đọc được repo**, và kẻ tấn công còn moi lại được từ lịch sử commit dù bạn đã xóa. Jenkins cung cấp **Credentials Store**: bạn lưu bí mật một lần, mã hóa, rồi tham chiếu bằng một **ID** không tiết lộ giá trị thật.

```
withCredentials([string(credentialsId: 'github-token', variable: 'TOKEN')]) {
    sh 'git clone https://$TOKEN@github.com/org/repo.git'
}
```

Ở đây **github-token** chỉ là cái tên; giá trị thật được Jenkins tiêm vào biến môi trường **TOKEN** lúc chạy và **không hiện trong log**. Đổi token chỉ cần sửa một chỗ trong Credentials Store, không phải lục lọi khắp code.

Webhook — vì sao tốt hơn polling? Có hai cách để Jenkins biết “có code mới”:

POLLING (hỏi liên tục) Jenkins → repo: "có gì mới?" repo → "chưa" Jenkins → repo: "có gì mới?" repo → "chưa" ... (mỗi phút, lãng phí) ...	WEBHOOK (được báo) Dev push code GitHub —POST→ Jenkins "nhánh main vừa được push!" Jenkins build NGAY
--	--

Polling nghĩa là Jenkins cứ vài phút lại hỏi Git “có gì mới không”. Cách này vừa **lãng phí tài nguyên** (99% lần hỏi nhận về “không có gì”), vừa **trễ** (code push xong phải chờ đến chu kỳ hỏi kế tiếp mới build). Webhook đảo ngược mô hình: **Git chủ động gọi Jenkins** ngay khoảnh khắc có push, qua

một HTTP POST. Kết quả: build kích hoạt tức thì, không có vòng hỏi vô ích.

Insight: Polling là “hỏi mãi”, webhook là “được gọi khi cần” — đây chính là nguyên tắc “*Don’t call us, we’ll call you*”. Bất cứ khi nào có thể để bên phát sự kiện chủ động thông báo, hãy chọn nó thay vì để bên nhận liên tục dò hỏi.

Ứng dụng AI trong session này

AI không thay bạn hiểu bản chất, nhưng tăng tốc rõ rệt ở các việc lặp và tra cứu:

- **Sinh và giải thích `Jenkinsfile`** : mô tả quy trình bằng tiếng Việt (“build Node, chạy test, deploy staging nếu là nhánh main”), để AI dựng khung Groovy, rồi bạn tự kiểm và hiểu từng `stage` .
- **Giải mã lỗi build**: dán log lỗi đỏ vào AI để nó chỉ ra nguyên nhân gốc và gợi ý cách sửa, thay vì mò trên Google.
- **Tra cứu plugin**: hỏi “plugin nào để tích hợp Jenkins với Slack/Docker” để rút ngắn thời gian tìm trong hơn 1800 plugin.

Lưu ý: AI có thể “chế” cú pháp Groovy không tồn tại. Luôn xác minh trên tài liệu chính thức và chạy thử — bạn chịu trách nhiệm cho pipeline, không phải AI.

Tóm tắt nhanh

- **CI/CD là chiến lược quản lý rủi ro**: tích hợp thật thường xuyên để integration hell không kịp hình thành, rút ngắn feedback loop.
- **CI → Continuous Delivery → Continuous Deployment** là ba mức trưởng thành: khác nhau ở chỗ ai bấm nút lên production và cần bao nhiêu test tự động.
- **Controller/Agent** tách điều phối khỏi thực thi để scale và cô lập môi trường; **lỗi nhỏ + plugin** để linh hoạt tích hợp mọi công cụ.
- **Pipeline (as Code)** thắng Freestyle vì đưa quy trình vào Git: version, review, rollback được.
- **Credentials** để không rò rỉ bí mật; **webhook** để build tức thì thay vì polling lãng phí.

Về nhà

Mục tiêu: cấu hình để **mỗi lần push code là Jenkins tự động build**.

1. Chạy Jenkins bằng Docker cho nhanh:

```
docker run -d --name jenkins -p 8080:8080 -p 50000:8080 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
# Lấy mật khẩu admin ban đầu:
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Vì sao có `initialAdminPassword` ? Đây là bước bảo mật: chỉ người **có quyền truy cập vào máy chủ** (đọc được file trong container) mới mở khóa được Jenkins lần đầu, tránh việc ai đó tình cờ vào được URL và chiếm quyền admin. Sau khi unlock, cài **Suggested plugins** và tạo **user admin** của riêng bạn.

2. Tạo Pipeline job, trở tới repo của bạn, và thêm một `Jenkinsfile` đơn giản vào repo (dùng ví dụ ở mục Pipeline).

3. Cấu hình webhook trên GitHub: vào repo → **Settings** → **Webhooks** → **Add webhook**:

- Payload URL: `http://<địa-chỉ-jenkins>:8080/github-webhook/`
- Content type: `application/json`
- Trigger: *Just the push event*
- Trong Jenkins job, bật **"GitHub hook trigger for GITScm polling"**.

4. Đảm bảo repo có `.gitignore` hợp lý để không đẩy rác lên (và không kích hoạt build vì file thừa):

```
node_modules/
dist/
.env
*.log
```

5. Push một commit bất kỳ và quan sát: Jenkins phải **tự động chạy build** trong vài giây. Nếu không chạy, kiểm tra mục *Recent Deliveries* của webhook trên GitHub để xem POST có tới Jenkins và nhận mã `200` không.

Nếu Jenkins chạy trên máy local, GitHub ngoài Internet không gọi vào được. Dùng `ngrok` `http 8080` để tạo một URL public tạm thời làm Payload URL.

Tự kiểm tra (câu hỏi “vì sao”)

1. Vì sao tích hợp code **thường xuyên** lại *giảm* đau thay vì *tăng* đau, dù mỗi lần vẫn phải merge?
2. Continuous Delivery và Continuous Deployment khác nhau ở đúng **một điểm** nào? Vì sao điểm đó lại đòi hỏi độ trưởng thành test khác nhau?
3. Vì sao Jenkins cố tình **không** để controller tự chạy build? Nêu ít nhất hai lý do.
4. Vì sao “lỗi nhỏ + plugin” lại giúp Jenkins sống sót qua nhiều năm khi hệ sinh thái công cụ thay đổi liên tục?
5. Vì sao đưa quy trình build vào **Jenkinsfile** trong Git lại an toàn hơn nhiều so với cấu hình Freestyle trên UI?
6. Vì sao hardcode token vào repo nguy hiểm ngay cả khi bạn đã xóa nó ở commit sau?
7. Vì sao webhook vừa nhanh hơn vừa tiết kiệm hơn polling? Mô tả bằng lời “ai gọi ai”.

Session 2

1h30

Jenkins

Pipeline

Jenkinsfile

Session 2 — Jenkins Pipeline & tự động hóa Build/Deploy

Mục tiêu: Viết được Jenkinsfile hoàn chỉnh: build → test → deploy.

Hãy tưởng tượng một job Jenkins được cấu hình hoàn toàn bằng UI: bạn vào trang web, click chuột, điền lệnh vào ô “Execute shell”, thêm plugin, kéo thả, lưu lại. Ban đầu mọi thứ chạy ngon. Nhưng ba tháng sau, không ai nhớ ai đã sửa gì. Server Jenkins hỏng ổ cứng, và toàn bộ cấu hình biến mất — vì nó nằm trong máy chủ, không nằm trong Git. Muốn biết “hôm qua ai đổi lệnh deploy”, bạn không có lịch sử. Muốn tạo một job giống hệt cho dự án mới, bạn phải click lại từ đầu và chắc chắn sẽ sót một ô nào đó.

Đây chính là “nỗi đau” mà Pipeline as Code sinh ra để giải quyết. Buổi này ta không chỉ học cú pháp Jenkinsfile, mà đào sâu **vì sao** từng lựa chọn tồn tại.

Vì sao “Pipeline as Code”

Cấu hình bằng UI có ba vấn đề cốt lõi:

1. **Không versioned** — thay đổi không có lịch sử, không biết ai sửa, không rollback được.
2. **Không tái tạo được** — job sống trong máy chủ Jenkins; mất máy là mất cấu hình.
3. **Không review được** — không ai có thể “duyet” một cú click chuột trước khi nó chạy trên production.

Pipeline as Code đảo ngược tất cả: bạn viết quy trình CI/CD thành một file text tên **Jenkinsfile**, đặt **ngay trong repo** cùng source code.

Vì sao đặt Jenkinsfile trong repo? Vì quy trình build phải tiến hóa cùng code. Khi bạn thêm một bước test mới ở nhánh feature, Jenkinsfile ở nhánh đó cũng thay đổi theo — và được merge cùng lúc với code. Cấu hình và code luôn khớp nhau, không bao giờ lệch pha.

Hệ quả trực tiếp:

- **Code review cho cả pipeline:** một PR sửa cách deploy sẽ được đồng nghiệp duyệt như mọi đoạn code khác.

- **Lịch sử đầy đủ:** `git blame Jenkinsfile` cho biết ai đổi lệnh deploy và vì sao.
- **Tái tạo tức thì:** dựng Jenkins mới, trở vào repo, pipeline chạy y hệt.
- **Tự phục vụ:** dev sửa được quy trình build mà không cần xin quyền admin Jenkins.

Declarative vs Scripted — chọn cái nào và vì sao

Jenkins có hai “phương ngữ” viết pipeline. Cả hai đều chạy trên Groovy, nhưng triết lý khác nhau.

Tiêu chí	Declarative	Scripted
Cú pháp	Có cấu trúc cố định (<code>pipeline {</code> <code>}</code>)	Groovy thuần, tự do
Độ dễ đọc	Cao, khai báo rõ ràng	Thấp hơn, giống code lập trình
Kiểm tra sớm (validation)	Có — báo lỗi cú pháp trước khi chạy	Ít — lỗi lộ ra lúc chạy
<code>post</code> , <code>when</code> , <code>environment</code>	Hỗ trợ sẵn dạng khai báo	Phải tự viết logic
Linh hoạt logic phức tạp	Hạn chế (nhưng đủ 95% nhu cầu)	Rất cao (vòng lặp, hàm, try/catch tùy ý)
Khuyến nghị	Mặc định nên dùng	Chỉ khi thật sự cần

Vì sao Declarative là lựa chọn mặc định? Vì đa số pipeline chỉ cần: chạy vài stage theo thứ tự, quản lý env, xử lý khi fail. Declarative diễn đạt đúng những thứ đó một cách rõ ràng và “chống sai sót” — Jenkins kiểm tra cú pháp trước khi chạy, nên bạn không đợi 10 phút build rồi mới biết gõ sai từ khóa.

Khi nào dùng **Scripted**? Khi bạn cần logic mà Declarative không diễn đạt nổi: vòng lặp động sinh stage từ danh sách, xử lý ngoại lệ phức tạp, tính toán runtime để quyết định luồng. Ngay cả khi đó, mẹo hay là dùng Declarative cho khung sườn và nhúng một khối `script { }` cho phần logic đặc biệt — giữ được sự rõ ràng mà vẫn linh hoạt.

Giải phẫu một Jenkinsfile (agent, stages, steps, environment, post)

Đây là một Jenkinsfile Declarative hoàn chỉnh cho một app Node.js — build, test, rồi deploy. Đọc qua trước, ta sẽ mổ xẻ từng phần bên dưới.

```

pipeline {
    // 1. agent: chạy pipeline ở đâu
    agent any

    // 2. environment: biến dùng chung cho mọi stage
    environment {
        APP_NAME      = 'my-web-app'
        DEPLOY_HOST    = 'deploy@10.0.0.20'
        // Lấy secret từ Jenkins Credentials - KHÔNG hardcode
        REGISTRY_CREDS = credentials('docker-registry')
    }

    options {
        timeout(time: 20, unit: 'MINUTES') // tránh build treo vô hạn
        disableConcurrentBuilds()          // không chạy 2 build chồng nhau
    }

    stages {
        stage('Build') {
            steps {
                echo "Building ${APP_NAME} ..."
                sh 'npm ci' // cài dependency đúng lockfile
                sh 'npm run build'
            }
        }

        stage('Test') {
            steps {
                sh 'npm test -- --ci --reporters=default --reporters=jest-junit'
            }
            post {
                always {
                    junit 'reports/junit.xml' // luôn thu kết quả test
                }
            }
        }

        stage('Deploy') {
            when {
                branch 'main' // chỉ deploy nhánh main
            }
            steps {
                withCredentials([sshUserPrivateKey(
                    credentialsId: 'deploy-ssh-key',
                    keyFileVariable: 'SSH_KEY'
                )]) {
                    sh '''
                        rsync -az -e "ssh -i $SSH_KEY" \
                            dist/ ${DEPLOY_HOST}:/var/www/${APP_NAME}/
                        ssh -i $SSH_KEY ${DEPLOY_HOST} \
                            "sudo systemctl restart ${APP_NAME}"
                    '''
                }
            }
        }
    }
}

```

```
// 3. post: chạy SAU khi các stage kết thúc
post {
    success {
        echo "✅ Build #${env.BUILD_NUMBER} thành công."
    }
    failure {
        echo "❌ Build #${env.BUILD_NUMBER} thất bại - xem log."
    }
    always {
        archiveArtifacts artifacts: 'dist/**', fingerprint: true
        cleanWs() // dọn workspace
    }
}
}
```

Giờ mở xẻ từng khối và **vì sao** nó có mặt:

agent — khai báo pipeline chạy ở đâu. **agent any** nghĩa là “bất kỳ máy nào rảnh”. Bạn có thể chỉ định **agent { label 'linux-docker' }** để buộc chạy trên node có công cụ phù hợp. *Vì sao cần?* Vì Jenkins thường là hệ thống nhiều node; bạn phải nói rõ môi trường thực thi, nếu không build có thể chạy nhầm chỗ thiếu công cụ.

environment — định nghĩa biến dùng chung. *Vì sao gom vào đây?* Để mọi stage dùng cùng một nguồn sự thật, tránh gõ lặp lại chuỗi **my-web-app** khắp nơi (đổi tên một chỗ là xong). Đặc biệt, **credentials('docker-registry')** là cách Jenkins tiêm secret an toàn — ta bàn kỹ ở mục sau.

stages → **stage** → **steps** — đây là xương sống. **stages** chứa nhiều **stage**; mỗi **stage** là một giai đoạn có tên (Build, Test, Deploy); trong mỗi **stage** là các **steps** — lệnh cụ thể. *Vì sao chia tầng như vậy?* Vì cái tên **stage** sẽ hiện thành từng ô màu trên giao diện Jenkins — nhìn một cái là biết build hỏng ở giai đoạn nào.

when — điều kiện chạy stage. **when { branch 'main' }** khiến stage Deploy chỉ chạy khi build nhánh **main**. *Vì sao?* Vì bạn build mọi nhánh để kiểm tra, nhưng chỉ deploy nhánh chính lên production.

post — khối chạy *sau* khi stages xong, phân theo kết quả:

- **always** : luôn chạy (thu artifact, dọn dẹp) — dù thành công hay thất bại.
- **success** : chỉ khi mọi thứ xanh.
- **failure** : chỉ khi có lỗi (gửi cảnh báo).

Vì sao `post` quan trọng đến vậy? Vì có những việc **bắt buộc phải làm dù build fail**: dọn workspace để không đầy ổ đĩa, thu kết quả test để phân tích, và — quan trọng nhất — **báo cho con người biết**. Nếu bạn nhét các lệnh này vào `steps`, chúng sẽ bị bỏ qua ngay khi một stage trước đó fail (pipeline dừng). `post` đảm bảo chúng luôn chạy.

Quản lý secret đúng cách (và vì sao)

Không bao giờ viết mật khẩu, token, private key trực tiếp vào Jenkinsfile. Lý do hiển nhiên: Jenkinsfile nằm trong Git — **hardcode secret** nghĩa là **rò rỉ vĩnh viễn** cho mọi người có quyền đọc repo, và secret còn nằm trong lịch sử Git ngay cả sau khi bạn xóa.

Jenkins cung cấp hai cơ chế:

1. `credentials()` trong `environment` — dạng ngắn gọn:

```
environment {
    REGISTRY_CREDS = credentials('docker-registry')
}
```

Với credential kiểu username/password, Jenkins tự tạo thêm `REGISTRY_CREDS_USR` và `REGISTRY_CREDS_PSW`.

2. `withCredentials()` — dạng tường minh, phạm vi hẹp:

```
withCredentials([usernamePassword(
    credentialsId: 'docker-registry',
    usernameVariable: 'REG_USER',
    passwordVariable: 'REG_PASS'
)]) {
    sh 'echo "$REG_PASS" | docker login -u "$REG_USER" --password-stdin registry.io'
}
```

Vì sao ưu tiên `withCredentials` ? Vì nó giới hạn secret trong đúng khối cần dùng — ra khỏi khối, biến biến mất. Phạm vi càng hẹp, rủi ro rò rỉ càng thấp.

Hai nguyên tắc sống còn:

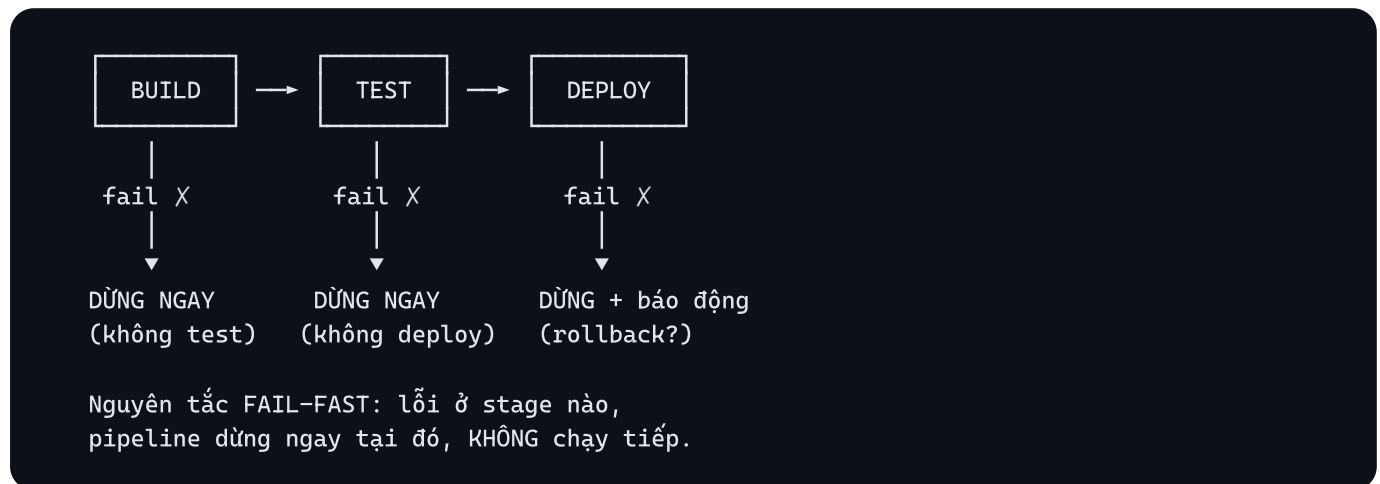
- **Vì sao không in secret ra log?** Vì log Jenkins thường được nhiều người xem và có thể được lưu trữ lâu dài. Một lệnh `echo $PASSWORD` vô tình biến secret thành công khai. Jenkins có che (mask) các biến credential trong log, nhưng đừng ỷ lại — dùng `--password-stdin` thay vì

truyền qua tham số dòng lệnh, và không bao giờ echo secret.

- **Vì sao không hardcode?** Vì secret trong Git là bất tử: xóa ở commit mới không xóa được ở lịch sử. Rò một lần là phải xoay (rotate) toàn bộ.

Vì sao phải chia stage: build → test → deploy

Bạn có thể nhét tất cả vào một stage khổng lồ chạy một script dài. Nhưng đừng. Hãy nhìn luồng chuẩn:



Vì sao tách stage là best practice:

- **Chẩn đoán tức thì:** Jenkins hiển thị từng stage thành ô riêng. Ô Test đỏ nghĩa là code build được nhưng test hỏng — bạn biết ngay lỗi ở đâu mà không phải đọc log dài dằng dặc.
- **Fail-fast tiết kiệm thời gian và ngăn thảm họa:** nếu Build fail, chạy Test làm gì? Nếu Test fail, tuyệt đối **không** Deploy. Việc tách stage khiến quy tắc “code hỏng không bao giờ lên production” được thực thi tự nhiên.
- **Đo lường được:** Jenkins ghi thời gian từng stage. Thấy stage Test ngày càng chậm, bạn biết cần tối ưu chỗ nào.

Vì sao thứ tự build → test → deploy là bất biến? Vì nó phản ánh mức độ tin cậy tăng dần. Build chứng minh “code biên dịch được”. Test chứng minh “code chạy đúng”. Chỉ khi qua cả hai cửa, ta mới cho phép Deploy chạm vào production. Đảo thứ tự này là mời gọi sự cố.

Artifact & notification — vì sao quan trọng

Artifact là sản phẩm build ra: file `.jar`, thư mục `dist/`, Docker image. `archiveArtifacts` lưu chúng lại gắn với số build.

Vì sao lưu artifact? Vì bạn cần **truy vết** và **khả năng rollback**. Khi production lỗi lúc 2h sáng, bạn muốn lấy đúng bản build #142 đã deploy để so sánh hoặc quay lui — chứ không phải build lại từ source (mà biết đâu source đã đổi). Artifact cộng với `fingerprint: true` cho bạn biết chính xác build nào tạo ra file nào, đi lên môi trường nào.

Notification là báo động khi có chuyện:

Vì sao cần báo khi fail? Vì một pipeline fail mà không ai biết thì vô dụng. Con người không ngồi nhìn Jenkins cả ngày. Nếu build `main` gãy lúc 3h chiều mà mãi 9h sáng hôm sau mới có người phát hiện, bạn đã mất nửa ngày với một nhánh chính hỏng — chặn cả team. Một tin nhắn Slack/email tức thì rút ngắn cách đó từ hàng giờ xuống hàng giây.

Ứng dụng AI: sinh nhanh Jenkinsfile

Viết Jenkinsfile từ đầu tốn thời gian nhớ cú pháp. AI (Claude, GitHub Copilot) rất giỏi sinh khung sườn — nhưng bạn phải **review**, không dán mù.

Một prompt tốt cần nêu đủ ngữ cảnh: ngôn ngữ, công cụ build, cách deploy, môi trường:

```
Viết một Jenkinsfile Declarative cho app Python (FastAPI):
- agent: node có label 'python3.11'
- Stage Build: tạo virtualenv, cài requirements.txt
- Stage Test: chạy pytest, xuất junit vào reports/
- Stage Deploy: chỉ nhánh main, build Docker image,
  push lên registry riêng (credential id 'my-registry'),
  rồi ssh vào server (credential 'prod-ssh') chạy docker compose up -d
- post: archive coverage report; failure thì báo Slack kênh #ci
- Dùng withCredentials, không hardcode secret, timeout 15 phút
```

Prompt càng cụ thể, output càng ít phải sửa. Nhưng **luôn review những điểm sau**:

- **Credential ID có đúng với Jenkins của bạn không?** AI đoán tên; bạn phải khớp với credential thật đã tạo.
- **Tên host, đường dẫn, label agent** — AI bịa giá trị placeholder; sửa cho đúng hạ tầng.
- **Lệnh deploy có an toàn không?** Đừng để AI sinh lệnh `rm -rf` hay `sudo` mà bạn không hiểu.
- **Secret có bị in ra log không?** Kiểm tra không có `echo $PASSWORD` lọt vào.
- **Logic when** — có chắc chỉ deploy nhánh production?

Vì sao phải review AI-generated pipeline kỹ hơn code thường? Vì Jenkinsfile có quyền chạm vào production và giữ secret. Một lỗi trong hàm tiện ích thì hỏng cục bộ; một lỗi trong pipeline có thể deploy nhầm, rò secret, hoặc xóa nhầm dữ liệu server. Coi output của AI như một PR từ đồng nghiệp mới: hữu ích, nhưng phải duyệt từng dòng.

Tóm tắt nhanh

- **Pipeline as Code** thắng cấu hình UI vì nó được versioned, review được, tái tạo được — đặt **Jenkinsfile** ngay trong repo để pipeline tiến hóa cùng code.
- **Declarative** là mặc định (rõ ràng, kiểm tra cú pháp sớm); **Scripted** chỉ khi cần logic phức tạp.
- Cấu trúc: **agent** (chạy ở đâu) → **environment** (biến chung) → **stages/steps** (việc cần làm) → **post** (dọn dẹp & báo động dù thành/bại).
- **Secret**: dùng **credentials()** / **withCredentials()**, không hardcode, không in ra log.
- Chia **build** → **test** → **deploy** để chẩn đoán nhanh và ép quy tắc fail-fast: code hỏng không bao giờ lên production.
- **Artifact** để truy vết & rollback; **notification** để con người biết ngay khi fail.

Về nhà

Mở rộng Jenkinsfile ở trên: thêm một bước **gửi thông báo Slack (hoặc email) khi build fail**. Gợi ý dùng khối **post { failure { ... } }**:

```
post {
    failure {
        // Cần cài Slack Notification plugin và cấu hình credential 'slack-token'
        slackSend(
            channel: '#ci-alerts',
            color: 'danger',
            message: "❌ ${env.JOB_NAME} #${env.BUILD_NUMBER} FAILED\n" +
                "Xem log: ${env.BUILD_URL}"
        )
        // Hoặc gửi email:
        // mail to: 'team@company.com',
        //      subject: "Build FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        //      body: "Chi tiết: ${env.BUILD_URL}"
    }
}
```

Yêu cầu: giải thích **vì sao** bạn đặt thông báo trong **failure** chứ không phải **always**, và **vì sao** message nên kèm **BUILD_URL**.

Tự kiểm tra (câu hỏi “vì sao”)

1. **Vì sao** đặt Jenkinsfile trong repo lại giúp cấu hình và code không bao giờ lệch pha?
2. **Vì sao** Declarative được khuyến nghị làm mặc định thay vì Scripted, dù Scripted linh hoạt hơn?
3. **Vì sao** các lệnh dọn dẹp và thu kết quả test nên nằm trong `post { always }` chứ không phải trong `steps` ?
4. **Vì sao** hardcoded secret vào Jenkinsfile nguy hiểm hơn nhiều so với hardcoded vào một file cấu hình local không đưa lên Git?
5. **Vì sao** thứ tự build → test → deploy phải bất biến, và điều gì xảy ra nếu ta cho phép deploy chạy song song với test?
6. **Vì sao** một pipeline fail mà không có notification thì gần như vô dụng với team?

Session 3

1h30

Ansible

Config Management

Session 3 — Ansible: Configuration Management

Mục tiêu: Dùng Ansible quản lý cấu hình và tích hợp vào Jenkins để deploy.

Hãy tưởng tượng: công ty bạn có 3 server web. Một ngày kia server thứ 2 bị lỗi lạ — trang trắng, còn hai server kia vẫn chạy ngon. Bạn SSH vào, mò mẫm cả buổi chiều mới phát hiện: cách đây 4 tháng, một đồng nghiệp đã “chữa cháy” bằng cách sửa tay file cấu hình Nginx trên đúng server đó, rồi... quên ghi lại. Không ai biết. Không có tài liệu. Server này giờ khác với hai server còn lại theo những cách không ai lường được.

Đó chính là **snowflake server** — một server “độc nhất vô nhị” như bông tuyết, không thể tái tạo, không ai dám đụng vào vì sợ nó sập. Và hiện tượng các server dần dần lệch khỏi trạng thái chuẩn theo thời gian gọi là **config drift**. Đây là cơn ác mộng kinh điển của vận hành hệ thống, và cũng chính là lý do Ansible ra đời.

Vì sao cần Configuration Management

Configuration Management (quản lý cấu hình) là việc **mô tả trạng thái mong muốn của hệ thống bằng code**, thay vì cấu hình thủ công bằng tay trên từng máy. Toàn bộ “server này phải cài Nginx, phải có file config như thế này, service phải đang chạy” được viết thành file text, lưu vào Git, và áp dụng tự động.

Vì sao điều này quan trọng đến vậy? Vì cấu hình thủ công có ba vấn đề chí tử:

- **Không tái lập được.** Bạn gõ 20 lệnh để dựng một server. Tháng sau cần dựng server thứ hai giống hệt — bạn có nhớ đủ 20 lệnh, đúng thứ tự, đúng tham số không? Gần như chắc chắn là không.
- **Config drift.** Mỗi lần ai đó “sửa nhanh một chút” trên production, khoảng cách giữa các server lớn dần. Sau vài tháng, không server nào giống server nào. Khi có sự cố, bạn không biết môi trường thật sự đang ở trạng thái nào.
- **Kiến thức nằm trong đầu người.** Nếu người dựng server nghỉ việc, cấu hình đó trở thành hộp đen.

Vì sao config drift nguy hiểm? Vì nó phá vỡ giả định nền tảng của mọi việc debug: “các môi trường giống nhau”. Khi staging khác production theo cách bạn không thấy được, bug sẽ chỉ xuất hiện trên production — đúng nơi bạn ít muốn nó xuất hiện nhất.

Configuration Management biến cấu hình thành **code có thể đọc, review, version, tái lập**. Server không còn là snowflake — nó là sản phẩm của một file mà ai cũng đọc được.

Idempotency: khái niệm quan trọng nhất và vì sao

Nếu chỉ được nhớ một khái niệm trong Session này, hãy nhớ **idempotency (tính bất biến khi lặp)**.

Idempotency nghĩa là: **chạy một thao tác nhiều lần cho ra cùng một kết quả như chạy một lần**. Ansible không mô tả “hãy làm việc X”, mà mô tả “trạng thái cuối cùng phải là Y”. Trước khi làm bất cứ gì, Ansible **kiểm tra trạng thái hiện tại**:

- Nếu hệ thống đã đúng trạng thái mong muốn → không làm gì cả (**ok**).
- Nếu chưa đúng → chỉ thực hiện đúng phần thay đổi cần thiết (**changed**).

Vì sao đây là khái niệm quan trọng nhất? Hãy so sánh với một script bash thông thường. Một script `apt install nginx && echo "... " >> /etc/nginx/nginx.conf` mà chạy hai lần sẽ ghi nội dung vào file **hai lần** — hỏng cấu hình. Bạn phải tự viết đủ thứ kiểm tra “nếu chưa có thì mới thêm”. Với Ansible, tính idempotent là **mặc định** — module tự lo phần đó.

Điều này mang lại một sự an tâm cực lớn: **bạn có thể chạy lại playbook bất cứ lúc nào mà không sợ hỏng**. Playbook trở thành “nguồn sự thật” (source of truth): muốn biết server phải trông như thế nào? Đọc playbook. Nghi server bị drift? Chạy lại playbook — nó sẽ kéo server về đúng chuẩn và cho bạn biết chính xác cái gì đã lệch.

Vì sao idempotency đánh bại config drift? Vì nó cho phép bạn **liên tục hội tụ về trạng thái chuẩn**. Chạy playbook theo lịch, và mọi thay đổi tay trái phép sẽ bị phát hiện (hiện **changed**) và tự động sửa lại. Snowflake không còn cơ hội hình thành.

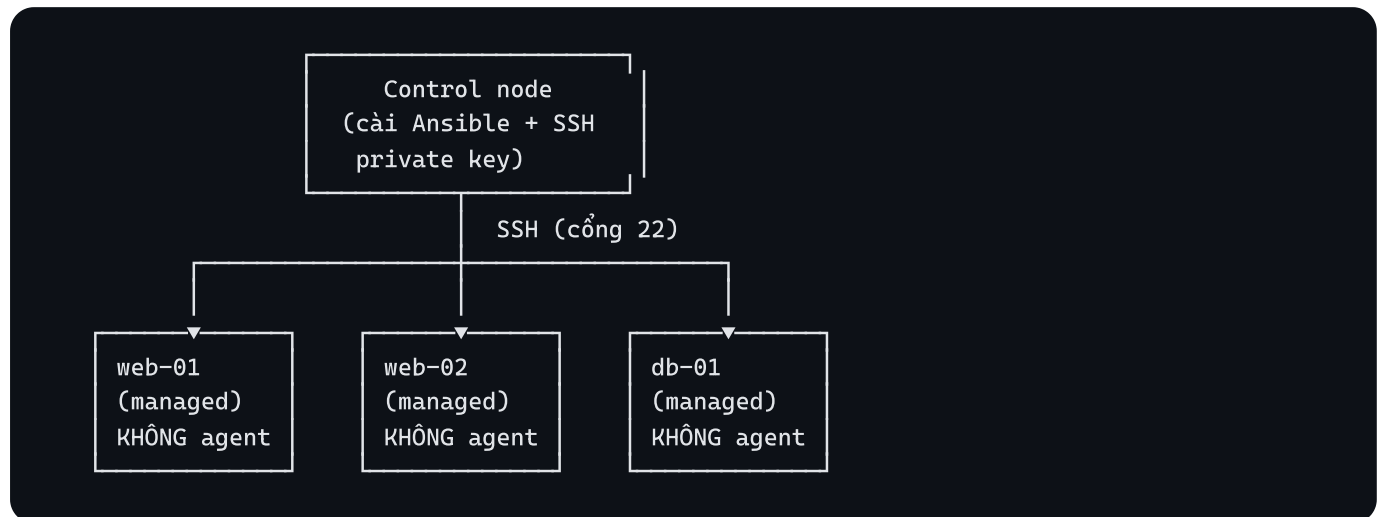
Ta sẽ thấy idempotency bằng mắt ở phần cuối, khi chạy playbook lần hai và nhận **changed=0**.

Kiến trúc agentless — vì sao đây là lợi thế

Ansible chạy theo mô hình **control node → managed nodes**:

- **Control node:** máy nơi bạn cài Ansible và chạy lệnh (laptop của bạn, hoặc một Jenkins agent).
- **Managed nodes:** các server bạn muốn cấu hình.

Ansible kết nối tới managed node qua **SSH** (giao thức có sẵn trên mọi máy Linux), đẩy các module Python nhỏ lên, chạy chúng, lấy kết quả rồi xóa đi. Không có tiến trình nào của Ansible ở lại trên server đích.



Đây gọi là kiến trúc **agentless** — không cần cài phần mềm thường trú (agent/daemon) trên máy đích. Điều này khác hẳn Puppet hay Chef, vốn yêu cầu cài agent trên mọi server và agent đó “phone home” về master theo chu kỳ.

Vì sao agentless là lợi thế lớn?

Tiêu chí	Agent-based (Puppet, Chef)	Agentless (Ansible)
Cài đặt ban đầu	Phải cài agent lên từng máy trước	Chỉ cần SSH + Python (thường có sẵn)
Bảo trì	Phải nâng cấp, giám sát, và agent	Không có agent để bảo trì
Bề mặt tấn công	Thêm 1 daemon mở cổng, chạy nền	Không thêm dịch vụ thường trú
Kiểm soát thời điểm chạy	Agent tự chạy theo chu kỳ	Bạn chủ động chạy (push) khi cần
Firewall	Cần mở cổng riêng cho agent	Dùng cổng SSH sẵn có

Vì sao “không agent” quan trọng về bảo mật? Mỗi daemon thường trú là một cửa sổ tấn công tiềm năng — một dịch vụ luôn chạy, luôn mở cổng, có thể có lỗ hổng. Ansible không thêm gì thường trú: khi không chạy playbook, trên server đích không có bất kỳ tiến trình Ansible nào tồn tại. Ít thành phần hơn = ít thứ có thể hỏng và bị khai thác hơn.

Lợi thế thực dụng nhất: **triển khai ngày đầu tiên gần như bằng không**. Có SSH là dùng được ngay, không cần “bootstrap agent” lên hàng trăm máy trước.

Các khái niệm cốt lõi: Inventory, Playbook, Task, Module, Role, Variables

Ansible xoay quanh vài khái niệm. Điều quan trọng không phải học thuộc tên, mà hiểu **vì sao mỗi thứ tồn tại**.

Inventory — LÀ GÌ: danh sách các máy đích, thường nhóm lại theo vai trò. **VÌ SAO:** để tách “làm gì” khỏi “làm ở đâu”. Cùng một playbook chạy được trên dev, staging, prod chỉ bằng cách đổi inventory.

```
# inventory.ini
[webservers]
web-01 ansible_host=10.0.0.11
web-02 ansible_host=10.0.0.12

[dbservers]
db-01 ansible_host=10.0.0.21

[all:vars]
ansible_user=deploy
ansible_ssh_private_key_file=~/.ssh/deploy_key
```

Playbook — LÀ GÌ: file YAML mô tả trạng thái mong muốn cho một nhóm máy. **VÌ SAO:** đây là “kịch bản” có thể đọc như tài liệu, review trong Git, và chạy lặp lại an toàn.

Task — LÀ GÌ: một bước trong playbook (ví dụ “đảm bảo Nginx đã cài”). **VÌ SAO:** chia công việc thành các đơn vị nhỏ, mỗi đơn vị idempotent độc lập.

Module — LÀ GÌ: đơn vị thực thi thật sự đằng sau mỗi task (`apt` , `copy` , `service` ...). **VÌ SAO:** module đã đóng gói sẵn logic kiểm tra trạng thái, nên task của bạn tự động idempotent mà không phải viết tay.

Variables — LÀ GÌ: giá trị có thể thay đổi (số worker, tên domain, phiên bản). **VÌ SAO:** để cùng một playbook thích ứng nhiều môi trường mà không sửa logic.

Role — LÀ GÌ: cách đóng gói playbook + biến + file + template thành một khối tái sử dụng theo cấu trúc thư mục chuẩn. **VÌ SAO:** để dùng lại (ví dụ role `nginx`) trên nhiều dự án, thay vì copy-paste.

Đây là một playbook cài và cấu hình Nginx:

```
# nginx.yml
- name: Cài đặt và cấu hình Nginx
  hosts: webserver
  become: true          # dùng sudo vì cài gói cần quyền root
  vars:
    worker_processes: 2

  tasks:
    - name: Đảm bảo gói nginx đã được cài
      apt:
        name: nginx
        state: present    # "present" = mong muốn có, không quan tâm cách
        update_cache: true

    - name: Đẩy file cấu hình từ template
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Reload nginx    # chỉ reload khi file THỰC SỰ đổi

    - name: Đảm bảo nginx đang chạy và tự khởi động cùng máy
      service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Reload nginx
      service:
        name: nginx
        state: reloaded
```

Chú ý cách viết: ta không nói “hãy cài”, mà nói **state: present** — mô tả kết quả mong muốn. Ta không nói “hãy restart”, mà dùng **notify** để handler chỉ chạy **khi cấu hình thực sự thay đổi**. Đó là idempotency thể hiện trong từng dòng.

Bây giờ, phần thú vị nhất. Chạy lần đầu:

```
$ ansible-playbook -i inventory.ini nginx.yml

PLAY [Cài đặt và cấu hình Nginx] *****
TASK [Đảm bảo gói nginx đã được cài] ***** changed
TASK [Đẩy file cấu hình từ template] ***** changed
TASK [Đảm bảo nginx đang chạy] ***** ok
RUNNING HANDLER [Reload nginx] ***** changed

PLAY RECAP *****
web-01 : ok=4  changed=2  unreachable=0  failed=0
```

Chạy lần hai — không đổi gì trên máy:

```
$ ansible-playbook -i inventory.ini nginx.yml

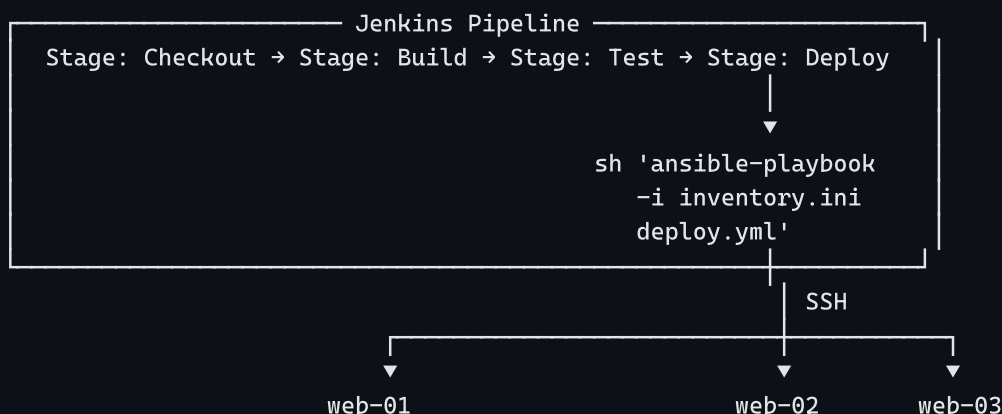
PLAY RECAP *****
web-01 : ok=3  changed=0  unreachable=0  failed=0
```

changed=0 . Ansible kiểm tra, thấy mọi thứ đã đúng trạng thái, và **không làm gì**. Handler reload cũng không chạy vì file config không đổi. Đây chính là idempotency bằng mắt thường: chạy bao nhiêu lần cũng an toàn, và số **changed** cho bạn biết chính xác server đã lệch bao nhiêu so với chuẩn.

Ansible ↔ Jenkins: ai làm gì và vì sao

Ở Session trước ta có Jenkins. Giờ ghép hai công cụ lại. Nguyên tắc cốt lõi là **phân vai rõ ràng**:

- **Jenkins điều phối (orchestration)**: quyết định *khi nào* deploy, *cái gì* deploy, chạy test, quản lý pipeline, phê duyệt, ghi log.
- **Ansible cấu hình (configuration)**: biết *làm thế nào* để đưa server về trạng thái đúng.



Trong **Jenkinsfile** , stage deploy chỉ đơn giản gọi Ansible:

```
stage('Deploy') {
    steps {
        sh 'ansible-playbook -i inventory/prod.ini nginx.yml'
    }
}
```

Vì sao phải tách “Jenkins điều phối / Ansible cấu hình”? Vì đó là hai loại trách nhiệm khác nhau, thay đổi vì lý do khác nhau. Logic “cấu hình server thế nào” nằm trong playbook — dùng được cả khi không có Jenkins (chạy tay lúc khẩn cấp, hay từ CI khác). Logic “quy trình duyệt và tự động hóa” nằm trong Jenkins. Gộp chung lại (nhét lệnh cấu hình vào script shell dài trong Jenkins) thì bạn mất tính tái lập, mất idempotency, và khóa chặt cấu hình vào một công cụ CI duy nhất.

Ứng dụng AI: sinh inventory đa môi trường

Một việc lặp đi lặp lại và dễ sai: dựng inventory cho nhiều môi trường (dev/staging/prod) với biến riêng từng môi trường. Đây là chỗ AI hỗ trợ tốt — bạn mô tả nhu cầu, AI sinh khung `group_vars` để bạn rà lại.

Ansible tự động nạp biến từ thư mục `group_vars/<tên_nhóm>.yaml`. Cấu trúc điển hình:

```
inventory/
├── dev.ini
├── staging.ini
├── prod.ini
├── group_vars/
│   ├── all.yaml          # biến chung mọi môi trường
│   ├── dev.yaml
│   ├── staging.yaml
│   └── prod.yaml
```

```
# group_vars/all.yaml - dùng chung
app_name: mytutor
nginx_port: 80

# group_vars/dev.yaml
env_name: dev
worker_processes: 1
domain: dev.mytutor.local
debug_mode: true

# group_vars/prod.yaml
env_name: prod
worker_processes: 4
domain: mytutor.com
debug_mode: false
```

Prompt gợi ý cho AI: “Sinh cho tôi cấu trúc `group_vars` cho ba môi trường dev/staging/prod của app Nginx, với `worker_processes` tăng dần và domain riêng từng môi trường, kèm file `all.yaml` cho biến chung.” AI cho ra khung nhanh, nhưng **bạn phải review**: giá trị prod (số worker, domain thật, tắt

debug) là thứ AI không thể tự biết đúng — nó chỉ dựng bộ khung để bạn điền và kiểm chứng.

Tóm tắt nhanh

- **Config drift** và **snowflake server** là hệ quả của cấu hình thủ công; Configuration Management biến cấu hình thành **code tái lập được**.
- **Idempotency** là khái niệm cốt lõi: mô tả *trạng thái mong muốn*, chạy lại bao nhiêu lần cũng an toàn, `changed=0` chứng minh hệ thống đã đúng chuẩn.
- Ansible **agentless** (SSH, không daemon thường trú): triển khai nhanh, ít bề mặt tấn công, không phải bảo trì agent.
- Khái niệm cốt lõi: **Inventory** (ở đâu), **Playbook/Task/Module** (làm gì và bằng cái gì), **Variables** (khác biệt giữa môi trường), **Role** (tái sử dụng).
- **Jenkins điều phối, Ansible cấu hình** — tách vai để giữ tính tái lập và không khóa vào một công cụ CI.

Về nhà

Đóng gói playbook Nginx ở trên thành một **Role tái sử dụng** tên `nginx`. Tạo cấu trúc thư mục chuẩn:

```
roles/
└─ nginx/
    ├── tasks/main.yml      # các task (từ playbook chuyển vào)
    ├── handlers/main.yml   # handler reload
    ├── templates/nginx.conf.j2
    └── defaults/main.yml   # biến mặc định (worker_processes ... )
```

Sau đó viết một playbook gọn chỉ gọi role:

```
- hosts: webservers
  become: true
  roles:
    - nginx
```

Mục tiêu: hiểu **vì sao** role giúp tái sử dụng — logic cấu hình Nginx giờ là một khối độc lập, dự án khác chỉ cần `roles: - nginx` là dùng lại được, không copy-paste.

Tự kiểm tra (câu hỏi “vì sao”)

1. Vì sao một script bash cài đặt thường **không** idempotent, còn playbook Ansible thì có? Điều gì trong module tạo ra khác biệt đó?
2. Vì sao config drift lại khiến việc debug production trở nên khó hơn nhiều so với khi mọi server giống nhau?
3. Vì sao kiến trúc agentless giảm bề mặt tấn công so với công cụ cần cài agent thường trú?
4. Vì sao ta tách bạch “Jenkins điều phối / Ansible cấu hình” thay vì nhét toàn bộ lệnh cấu hình vào một script shell trong Jenkins?
5. Khi chạy playbook lần hai và thấy `changed=0`, điều đó **chứng minh** điều gì về trạng thái server?

Session 4

1h30

IaC

Terraform

GitOps

Session 4 — Configuration as Code: hệ sinh thái công cụ

Mục tiêu: Nắm bức tranh tổng thể 'everything as code' và biết chọn tool phù hợp.

Hãy tưởng tượng bạn dựng một server bằng tay: SSH vào máy, gõ hàng chục lệnh cài đặt, chỉnh vài file cấu hình, mở vài cổng tường lửa. Server chạy tốt. Ba tháng sau nó "chết", hoặc sếp bảo dựng thêm một server y hệt cho môi trường staging. Bạn ngồi nhớ lại: mình đã gõ những lệnh gì? Đã sửa dòng nào trong file nào? Không ai nhớ nổi. Kết quả là server mới "hao hao" server cũ nhưng khác ở vài chỗ bí ẩn, và những khác biệt đó chính là nguồn gốc của câu nói kinh điển: "trên máy tôi chạy mà".

Vấn đề cốt lõi: **hạ tầng dựng bằng tay không lặp lại được, không kiểm tra được, không rollback được**. Cả buổi học này trả lời cho câu hỏi làm sao thoát khỏi cái bẫy đó, và vì sao ngành công nghiệp đã đồng thuận trên một câu trả lời: biến mọi thứ thành **code**.

Vì sao "Everything as Code"

Ý tưởng "everything as code" nghĩa là: thay vì thao tác thủ công lên hệ thống, bạn **mô tả trạng thái mong muốn của hệ thống bằng file text**, rồi để công cụ đọc file đó và tự dựng ra hệ thống. Hạ tầng (server, mạng, database) trở thành Infrastructure as Code (IaC). Cấu hình phần mềm trở thành Configuration as Code. Môi trường chạy ứng dụng trở thành container as code. Quy trình triển khai trở thành pipeline as code.

Vì sao đây là bước ngoặt? Vì khoảnh khắc một thứ trở thành file text, nó lập tức thừa hưởng toàn bộ "vũ khí" mà ngành phần mềm đã mài giũa suốt 30 năm cho việc quản lý code:

- **Git:** mọi thay đổi có lịch sử, biết ai sửa, sửa gì, lúc nào.
- **Code review:** một thay đổi hạ tầng phải qua Pull Request, có người thứ hai duyệt trước khi áp dụng.
- **CI/CD:** thay đổi được test tự động trước khi chạm production.
- **Tái sử dụng:** copy một file, đổi vài tham số là có môi trường mới.

Vì sao quan trọng: Khi hạ tầng là code, một sai lầm nguy hiểm ("xóa nhầm database production") không còn là một cú gõ Enter không thể thu hồi, mà là một dòng diff trong Pull Request mà đồng nghiệp có cơ hội chặn lại trước khi nó xảy ra.

Ba nguyên tắc: versioned, declarative, reproducible

Everything as code chỉ thực sự có giá trị khi tuân theo ba nguyên tắc. Điều quan trọng không phải là học thuộc tên, mà là hiểu **mỗi nguyên tắc giải quyết nỗi đau cụ thể nào**.

Versioned (có phiên bản). Cấu hình được lưu trong Git. Vì sao cần? Vì khi production sập lúc 2 giờ sáng, câu hỏi đầu tiên luôn là "vừa nãy có gì thay đổi?". Nếu hạ tầng là code trong Git, bạn `git log` ra ngay thủ phạm và `git revert` để **rollback** về trạng thái tốt cuối cùng. Không có versioning, bạn chỉ có thể đoán mò.

Declarative (khai báo). Bạn mô tả *cái bạn muốn* ("tôi muốn 3 server chạy nginx"), không phải *các bước để đạt được nó* ("cài nginx, rồi start service, rồi..."). Vì sao tốt hơn? Vì công cụ tự so sánh trạng thái hiện tại với trạng thái mong muốn và chỉ làm phần còn thiếu. Chạy lại 10 lần vẫn ra đúng một kết quả — tính chất gọi là **idempotent**. Cách làm thủ công (imperative) thì chạy lần hai thường hỏng: "port đã bị chiếm", "user đã tồn tại".

Reproducible (tái lập được). Từ cùng một file, bất kỳ ai, bất kỳ lúc nào, cũng dựng ra hệ thống giống hệt. Vì sao đây là mục tiêu tối thượng? Vì nó tiêu diệt tận gốc "works on my machine". Nếu dev, staging, production đều sinh ra từ cùng một mô tả code, thì sự khác biệt môi trường — nguyên nhân của vô số bug khó chịu — biến mất.

Vì sao quan trọng: Ba nguyên tắc này không rời rạc. Versioned cho bạn lịch sử, declarative cho bạn tính nhất quán, reproducible cho bạn sự tin cậy. Thiếu một cái, hai cái còn lại yếu đi rõ rệt.

Provisioning vs Configuration: Terraform và Ansible khác nhau chỗ nào

Đây là chỗ người mới hay nhầm nhất, nên cần phân biệt rạch ròi bằng một phép ẩn dụ: xây nhà.

Provisioning là *dựng cái nhà lên*: mua đất, đổ móng, xây tường — tức là tạo ra hạ tầng chưa từng tồn tại (server ảo, mạng, load balancer, database trên cloud). **Terraform** là công cụ tiêu biểu cho việc này.

Configuration management là *bài trí bên trong ngôi nhà đã có*: sơn tường, lắp đồ đạc, cài phần mềm — tức là cấu hình những máy **đã tồn tại**. **Ansible** (đã học ở Session 3) là công cụ tiêu biểu.

Vì sao cần tách ra hai loại? Vì hai bài toán có bản chất khác nhau. Terraform cần biết “trên cloud hiện có những resource nào của tôi” để không tạo trùng — nên nó cần **state** (xem mục sau). Ansible thường chạy trên máy đã có sẵn, thao tác qua SSH, và mỗi task được thiết kế idempotent nên không cần state phức tạp.

Ranh giới trong thực tế: Terraform tạo ra 3 con server trống trên AWS, rồi “bàn giao” danh sách IP cho Ansible; Ansible SSH vào từng con để cài nginx, deploy app. Nhiều đội dùng đúng cả hai theo cách phân công này, chứ không phải chọn một bỏ một.

Vì sao Terraform cần “state” (và OpenTofu là gì)

Terraform lưu một file gọi là **state** (thường là `terraform.tfstate`). Đây là điểm gây bối rối nhất, nên hãy hiểu **vì sao nó bắt buộc phải tồn tại**.

Terraform là declarative: bạn viết “tôi muốn 1 server”. Nhưng để biết *phải làm gì* khi bạn chạy `terraform apply`, nó cần trả lời câu hỏi: “server đó đã tồn tại chưa?”. Cloud không có cách nào để Terraform hỏi “cái server mà tôi mô tả trong file này là con nào trong hàng nghìn server trên tài khoản?”. Vì vậy Terraform tự **ghi sổ**: state file là cuốn sổ ánh xạ “resource trong code của tôi” ↔ “ID thật trên cloud”.

Nhờ state, khi bạn đổi “1 server” thành “2 server”, Terraform biết đã có 1 con rồi nên chỉ tạo thêm 1 con nữa (chứ không xóa làm lại). State cũng cho phép Terraform **phát hiện drift**: nếu ai đó sửa tay resource trên cloud, state khác thực tế, Terraform báo cho bạn.

Vì sao quan trọng: State chứa thông tin nhạy cảm (đôi khi cả password, private key) và là “nguồn sự thật” của hạ tầng. Mất state = Terraform “mất trí nhớ”, không còn biết nó đã quản lý gì. Vì vậy state production luôn được lưu ở **remote backend** (S3, GCS...) có khóa và mã hóa, không bao giờ commit vào Git.

OpenTofu là gì? Năm 2023, HashiCorp đổi giấy phép Terraform từ mã nguồn mở (MPL) sang BSL — hạn chế hơn. Cộng đồng lo ngại nên **fork** Terraform ra thành **OpenTofu**, đặt dưới Linux Foundation, giữ giấy phép mở hoàn toàn. Về cú pháp OpenTofu gần như tương thích Terraform; bạn nên biết nó tồn tại để không bị bất ngờ khi thấy team dùng lệnh `tofu` thay cho `terraform`.

Một ví dụ tối giản Terraform tạo một máy ảo:

```
# main.tf - mô tả TRẠNG THÁI MONG MUỐN, không phải các bước
resource "aws_instance" "web" {
  ami          = "ami-0abc12345"    # ảnh hệ điều hành
  instance_type = "t3.micro"         # cấu hình máy

  tags = {
    Name = "web-server-01"
    Env  = "staging"
  }
}
```

Giải thích: bạn không viết “gọi API tạo máy”. Bạn khai báo “tồn tại một `aws_instance` tên web như thế này”. Chạy `terraform apply`, Terraform so sánh với state: chưa có thì tạo, có rồi thì bỏ qua, khác thì sửa. Đó chính là declarative + idempotent thể hiện bằng code.

Docker Compose: môi trường như một file

Terraform lo hạ tầng ở tầng cloud. Nhưng còn “môi trường chạy ứng dụng” ngay trên một máy — app cần Node, cần Postgres, cần Redis? Đây là đất diễn của **Docker** và **Docker Compose**.

Docker đóng gói ứng dụng cùng mọi thứ nó cần vào một **container** — một hộp cô lập chạy giống hệt nhau ở mọi nơi. Điều này trực tiếp hiện thực hóa nguyên tắc *reproducible*: container chạy trên máy bạn và trên production là **cùng một image**, nên không còn “works on my machine”.

Docker Compose đi xa hơn: nó mô tả *nhiều container liên kết với nhau* trong một file YAML duy nhất. Toàn bộ môi trường dev — app + database + cache — trở thành **một file bạn có thể commit vào Git**.

```
# docker-compose.yml - toàn bộ môi trường trong một file
services:
  app:
    build: .                # build image từ Dockerfile trong thư mục này
    ports:
      - "3000:3000"        # map cổng máy thật → cổng container
    depends_on:
      - db                 # app khởi động sau db

  db:
    image: postgres:16     # dùng image Postgres có sẵn
    environment:
      POSTGRES_PASSWORD: dev_only_secret
    volumes:
      - db_data:/var/lib/postgresql/data # dữ liệu bền, không mất khi tắt container

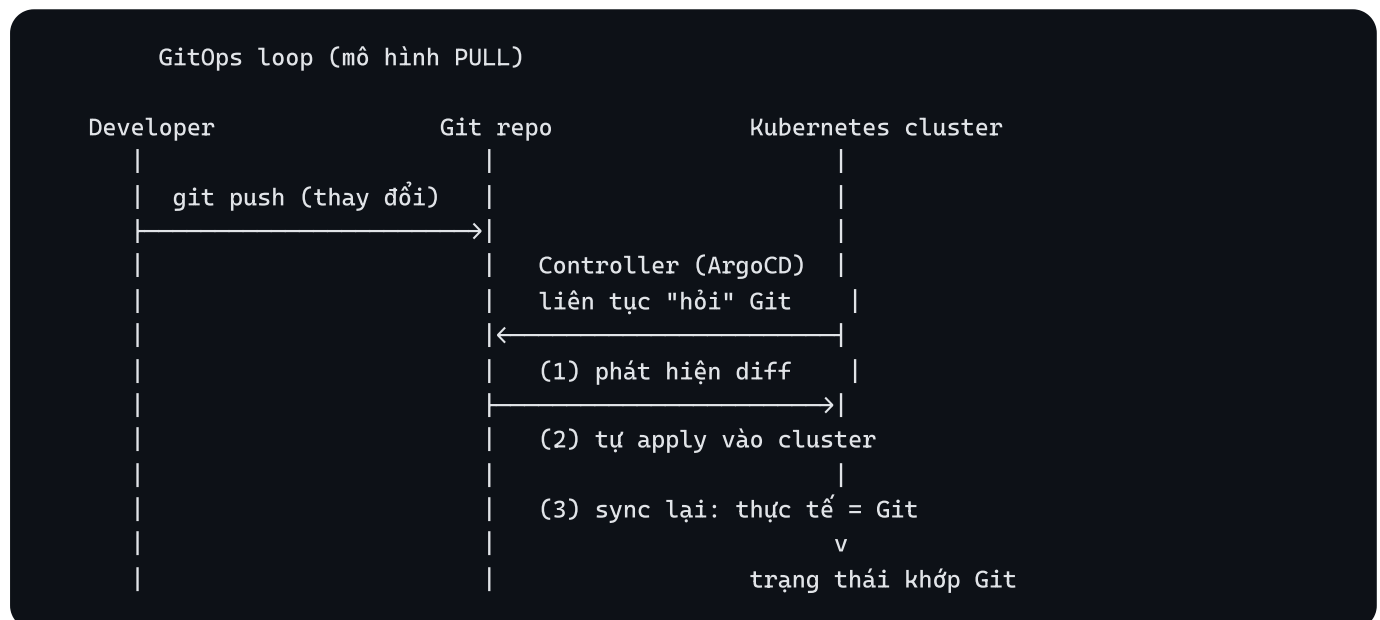
volumes:
  db_data:
```

Giải thích: một lệnh `docker compose up` dựng cả app lẫn database, nối mạng cho chúng, đúng thứ tự. Thành viên mới vào team chỉ cần `git clone` rồi `docker compose up` là có môi trường y hệt mọi người — thay cho một trang wiki dài “cách cài môi trường” mà luôn lỗi thời. Đó là “environment as code”.

GitOps: vì sao lấy Git làm nguồn sự thật

GitOps đẩy triết lý này tới cực hạn cho việc **triển khai** (deployment), thường trên Kubernetes.

Nguyên tắc: **Git repository là nguồn sự thật duy nhất (single source of truth) cho trạng thái mong muốn của hệ thống**. Muốn thay đổi production? Bạn không SSH vào cluster, không gõ `kubectl apply` bằng tay. Bạn **commit vào Git**. Một phần mềm chạy trong cluster (ArgoCD hoặc Flux) tự động kéo thay đổi về và áp dụng.



Vì sao đây là ý tưởng mạnh?

- **Nguồn sự thật rõ ràng**: muốn biết production đang chạy gì, đọc Git, không cần đăng nhập vào cluster.
- **Tự phục hồi (self-healing)**: nếu ai đó sửa tay cluster, controller thấy thực tế lệch Git và **tự kéo về đúng như Git** — drift bị triệt tiêu tự động.
- **Audit & rollback miễn phí**: mọi thay đổi production là một commit. Rollback = `git revert`. Ai đổi gì lúc nào — Git đã ghi sẵn.
- **Bảo mật tốt hơn**: không ai cần quyền truy cập trực tiếp vào cluster; họ chỉ cần quyền push vào Git.

Vì sao quan trọng: GitOps biến câu hỏi “làm sao triển khai an toàn?” thành câu hỏi đã được giải quyết từ lâu “làm sao merge code an toàn?”. Toàn bộ quy trình review, approval, CI của code tự động áp dụng cho hạ tầng.

Bổ sung nhanh — ba công cụ hay gặp:

- **Vagrant:** dựng máy ảo (VM) dev đồng nhất từ một **Vagrantfile**. Vai trò giống Compose nhưng ở tầng VM thay vì container; ngày nay ít dùng hơn.
- **Packer:** “nướng” (bake) sẵn một image máy (AMI, image Docker) đã cài đặt đầy đủ, để lúc provisioning khởi động cực nhanh. Nó tạo ra *nguyên liệu*, còn Terraform *dùng* nguyên liệu đó.
- **Helm:** “trình quản lý gói” cho Kubernetes, đóng gói nhiều file manifest thành một “chart” có tham số. Thường dùng chung với GitOps.

Imperative vs Declarative, Push vs Pull — vì sao xu hướng dịch chuyển

Có hai trục tư duy xuyên suốt cả hệ sinh thái.

Imperative vs Declarative — *cách bạn mô tả điều mong muốn*. Imperative liệt kê từng bước (“làm A, rồi B, rồi C”); một script bash cài đặt là imperative. Declarative mô tả kết quả (“hệ thống phải trông thế này”) và để công cụ tự tính các bước. Xu hướng nghiêng hẳn về declarative vì nó **idempotent** (chạy lại an toàn), **tự tính phần chênh lệch**, và **dễ đọc** — file mô tả *cái gì* dễ hiểu hơn script mô tả *làm sao*.

Push vs Pull — *ai khởi động việc áp dụng thay đổi*.

PUSH model
(Ansible, kubectl apply)

```
[ máy điều khiển ]
| ĐẨY cấu hình
| ra ngoài
v
[ server / cluster ]
```

- Cần quyền vào đích
- Áp dụng 1 lần khi bạn chạy

PULL model
(ArgoCD, Flux)

```
[ Git repo ]
^ agent trong hệ thống
| TỰ KÉO về
|
[ agent ] → [ cluster ]
```

- Đích tự lấy, không mở quyền vào
- Liên tục đồng bộ, tự sửa drift

Trong **push**, một máy trung tâm đẩy cấu hình ra các đích — nó phải có quyền truy cập vào từng đích, và thay đổi chỉ xảy ra *khi bạn bấm chạy*. Trong **pull**, một agent nằm *bên trong* hệ thống tự kéo cấu hình từ nguồn sự thật về và liên tục đồng bộ.

Vì sao **declarative + pull** đang thắng thế? Vì cặp đôi này cho **tự phục hồi liên tục**: hệ thống không chỉ đúng lúc bạn deploy mà luôn được kéo về đúng trạng thái mong muốn; và **bảo mật tốt hơn** vì không cần mở quyền truy cập từ ngoài vào — agent chủ động lấy về. Push vẫn hữu ích cho tác vụ một lần (chạy migration, cấu hình máy vật lý), nên đừng coi đây là “cái này thay thế cái kia” mà là “mỗi cái hợp một cảnh”.

Bản đồ chọn tool (khi nào dùng cái nào)

Công cụ	Nhóm	Declarative?	Khi nào dùng
Terraform / OpenTofu	Provisioning hạ tầng (IaC)	Có (có state)	Tạo/quản lý server, mạng, DB, resource trên cloud
Ansible	Configuration management	Chủ yếu (module idempotent)	Cài phần mềm, cấu hình OS trên máy đã tồn tại
Docker	Containerized environment	Có (Dockerfile)	Đóng gói 1 ứng dụng chạy giống nhau mọi nơi
Docker Compose	Multi-container local env	Có (YAML)	Dựng môi trường dev nhiều dịch vụ trên 1 máy
Kubernetes	Điều phối container	Có (manifest)	Chạy container quy mô lớn, tự scale, tự phục hồi
ArgoCD / Flux (GitOps)	Continuous deployment	Có	Lấy Git làm nguồn sự thật cho deploy lên K8s
Helm	Đóng gói app K8s	Có (chart)	Quản lý bộ manifest phức tạp có tham số
Packer	Xây image máy	Có (template)	Bake sẵn AMI/image để khởi động nhanh
Vagrant	Máy ảo dev	Có (Vagrantfile)	Môi trường dev đồng nhất ở tầng VM

Cách đọc bản đồ: hỏi “mình đang ở tầng nào?”. Tạo *hạ tầng* → Terraform. Cấu hình *máy đã có* → Ansible. Đóng gói *một app* → Docker. Dựng *môi trường dev nhiều dịch vụ* → Compose. *Triển khai liên tục* lên K8s → GitOps. Chúng bổ trợ nhau nhiều hơn là cạnh tranh.

Ứng dụng AI

Trợ lý AI đặc biệt hợp với “everything as code” vì đầu ra là file text có cấu trúc rõ, cú pháp nhất quán. Bạn có thể nhờ AI **sinh khung (scaffold)** Terraform hoặc docker-compose từ một mô tả bằng tiếng Việt.

Cách prompt hiệu quả — nêu rõ *ý định, ràng buộc, provider/phiên bản*:

“Sinh cho tôi file Terraform tạo 2 EC2 t3.micro trên AWS region ap-southeast-1, gắn security group chỉ mở cổng 22 và 443, tag Env=staging. Dùng biến cho instance_type. Chỉ trả code HCL, giải thích ngắn từng block.”

Nhưng — và đây là phần quan trọng nhất — **AI sinh khung, con người phải review**. Cần soi kỹ:

- **State & backend:** AI hay quên cấu hình remote backend, hoặc gợi ý commit state vào Git. Tuyệt đối kiểm tra state production nằm ở remote backend có mã hóa, không lọt vào repo.
- **Secret:** AI thường viết thẳng password/API key vào file (như `POSTGRES_PASSWORD` ví dụ trên chỉ để demo local). Trên thật phải dùng biến môi trường, secret manager, hoặc `.tfvars` không commit. Quét kỹ mọi chuỗi trông giống bí mật.
- **Phạm vi quyền & mạng:** AI hay mở rộng cổng (`0.0.0.0/0`), cấp quyền IAM quá rộng cho “chạy được”. Thu hẹp lại đúng nhu cầu.
- **Chi phí & phiên bản:** kiểm tra instance type, region, và version image không phải loại đắt đỏ hay đã lỗi thời.

Vì sao quan trọng: Một file IaC sai có thể xóa nhầm database hoặc phơi bày hạ tầng ra Internet. AI tăng tốc phần gõ code, nhưng trách nhiệm về *hậu quả* vẫn thuộc người bấm `apply`. Hãy coi output của AI như PR của một junior: hữu ích, nhưng bắt buộc review.

Tóm tắt nhanh

- “Everything as code” biến hạ tầng/cấu hình/môi trường thành file text để thừa hưởng Git, review, CI/CD.
- Ba nguyên tắc: **versioned** (rollback được), **declarative** (idempotent, tả *cái gì*), **reproducible** (diệt “works on my machine”).
- **Provisioning** (Terraform) dựng hạ tầng mới; **configuration** (Ansible) cấu hình máy đã có — hai bài toán khác nhau.

- Terraform cần **state** để ánh xạ code ↔ resource thật; **OpenTofu** là bản fork mã nguồn mở của Terraform.
- **Docker Compose** = cả môi trường dev trong một file; **GitOps** = Git là nguồn sự thật, controller tự pull và self-heal.
- Xu hướng nghiêng về **declarative + pull** vì tự phục hồi liên tục và bảo mật tốt hơn.

Về nhà

Phác thảo (trên giấy hoặc trong một file README) một pipeline GitOps đơn giản cho một app nhỏ:

1. Repo `app-config` chứa các manifest Kubernetes (hoặc `docker-compose.yml`) mô tả trạng thái mong muốn.
2. Bạn tạo nhánh, sửa số replica từ 2 lên 3, mở **Pull Request**.
3. Đồng đội **review** diff, approve, merge vào `main`.
4. Controller (ArgoCD/Flux) **phát hiện** commit mới trên `main`, tự **apply** vào cluster.
5. Nếu có sự cố: `git revert` commit đó — hệ thống tự kéo về trạng thái cũ.

Hãy viết ra ba câu trả lời: (a) vì sao bước review ở #3 là “lá chắn” quan trọng nhất; (b) chuyện gì xảy ra nếu ai đó sửa tay cluster ở #4; (c) secret (mật khẩu DB) nên nằm ở đâu trong sơ đồ này và vì sao *không* để trong repo config.

Tự kiểm tra (câu hỏi “vì sao”)

1. Vì sao “hạ tầng dựng bằng tay” lại sinh ra vấn đề “works on my machine”, và mỗi nguyên tắc trong ba nguyên tắc (versioned/declarative/reproducible) chặn nó ở đâu?
2. Vì sao Terraform *bắt buộc* phải có state, trong khi Ansible phần lớn không cần? Điều đó liên quan gì tới khác biệt provisioning vs configuration?
3. Vì sao OpenTofu ra đời — và vì sao một sự thay đổi *giấy phép* lại đủ khiến cộng đồng fork cả một công cụ?
4. Vì sao mô hình **pull** của GitOps vừa an toàn hơn vừa cho khả năng tự phục hồi so với việc `kubectl apply` bằng tay (push)?
5. Vì sao khi để AI sinh IaC, hai thứ *state* và *secret* lại là điểm phải review kỹ nhất?

Session 5 — Golang để viết DevOps tool

Mục tiêu: Hiểu vì sao Go phổ biến trong DevOps và tự viết được 1 CLI tool nhỏ.

Hãy thử liệt kê những công cụ hạ tầng bạn dùng hằng ngày: Docker, Kubernetes, Terraform, Prometheus, Helm, containerd, etcd, Consul, Vault, Grafana Loki... Có một điểm chung kỳ lạ: gần như **tất cả** đều được viết bằng Go. Đây không phải trùng hợp, cũng không phải vì lập trình viên DevOps "thích" Go. Đó là kết quả của một loạt quyết định kỹ thuật rất cụ thể — và khi hiểu được *vì sao*, bạn sẽ hiểu luôn tại sao Go gần như là lựa chọn mặc định khi cần viết một tool hạ tầng mới.

Bài này không dạy bạn "cú pháp Go từ A đến Z". Mục tiêu là: hiểu **vì sao** Go hợp với tooling, rồi tự tay viết được một CLI tool nhỏ chạy thật.

Vì sao Go thống trị DevOps tooling

Câu hỏi cốt lõi: một *tool hạ tầng* cần gì ở ngôn ngữ? Nó khác với một web app. Nó thường phải: chạy trên máy người khác (server, CI runner, laptop dev), khởi động nhanh, gọi nhiều I/O (mạng, disk, process con), và **đễ phân phối** cho hàng nghìn người với đủ loại HĐH. Go được thiết kế trùng gần như mọi tiêu chí đó.

- **Compile ra một binary tĩnh duy nhất.** Không cần cài Python, Node, JVM hay bất kỳ runtime nào trên máy đích. Bạn đưa cho ai đó *một file*, họ chạy được ngay. Với người phân phối tool, đây là khác biệt sống còn (mục sau sẽ đào sâu).
- **Cross-compile cực dễ.** Từ một máy Mac, bạn build ra binary cho Linux/Windows/ARM chỉ bằng vài biến môi trường. Không cần dựng máy ảo cho từng HĐH.
- **Concurrency rẻ và đơn giản.** Goroutine cho phép gọi song song hàng trăm request I/O mà không phải vật lộn với thread pool hay callback hell. Tool hạ tầng vốn "chờ I/O" rất nhiều — điều này hợp một cách tự nhiên.
- **Hiệu năng cao, gần với native.** Nhanh hơn hẳn ngôn ngữ thông dịch, đủ để chạy trong vòng lặp nóng của scheduler như Kubernetes.
- **Cú pháp nhỏ, dễ đọc, dễ review.** Tool hạ tầng thường là mã nguồn mở, được nhiều người đóng góp. Ngôn ngữ càng ít "phép thuật" thì càng dễ đọc code người lạ.

Vì sao điều này quan trọng? Google tạo ra Go chính để giải bài toán nội bộ của họ: build hệ thống lớn với đội ngũ đông, biên dịch nhanh, deploy đơn giản. Tình cờ, đó cũng đúng là bài toán của cả ngành DevOps. Docker (2013) chọn Go và tạo hiệu ứng lan tỏa — hệ sinh thái quanh nó cũng chọn Go để tương thích. Thành công của Go trong lĩnh vực này vừa là kỹ thuật, vừa là hiệu ứng mạng.

Binary tĩnh & cross-compile: siêu năng lực phân phối

Hãy so sánh cảm giác thực tế. Bạn viết một tool bằng Python và muốn đồng nghiệp dùng: họ phải cài đúng phiên bản Python, tạo virtualenv, `pip install` một mớ dependency, và cầu nguyện không xung đột version. Với Node cũng tương tự với `node_modules`. Runtime và dependency trở thành gánh nặng của *người dùng*.

Go lật ngược điều đó. Trình biên dịch gói toàn bộ code của bạn **và** các thư viện phụ thuộc vào **một file thực thi duy nhất**. Người dùng chỉ cần:

```
./mytool
```

Không cài runtime. Không có `node_modules`. Không lo “máy tôi thiếu thư viện”. Đây là lý do bạn có thể `curl` một binary Go về rồi chạy ngay — cách phân phối phổ biến của rất nhiều DevOps tool.

Vì sao “binary tĩnh” là siêu năng lực? Vì nó dời toàn bộ độ phức tạp của việc quản lý phụ thuộc từ *thời điểm chạy* (trên máy người dùng, nơi bạn không kiểm soát được) sang *thời điểm build* (trên máy bạn, nơi bạn kiểm soát hoàn toàn). Một tool “chỉ cần một file” là một tool mà support ticket giảm đi một nửa.

Cross-compile là phần thứ hai của siêu năng lực. Trình biên dịch Go biết cách sinh mã cho nhiều cặp HĐH/kiến trúc, nên từ *một máy* bạn build được cho *mọi máy* (chi tiết ở mục GOOS/GOARCH cuối bài).

Error handling kiểu Go — vì sao tường minh lại tốt cho tool

Nhiều người mới đến từ Java/Python thấy khó chịu với Go: không có `try/catch`, mà hàm nào có thể lỗi thì trả về thêm một giá trị `error`, và bạn phải viết `if err != nil` khắp nơi. Trông dài dòng. Nhưng với *tool hạ tầng*, đây lại là một lựa chọn thiết kế đúng.

```
resp, err := http.Get(url)
if err != nil {
    return fmt.Errorf("gọi %s thất bại: %w", url, err)
}
defer resp.Body.Close()
```

Vì sao cách này tốt cho tooling?

- **Lỗi là giá trị, không phải luồng điều khiển ẩn.** Với exception, một lỗi có thể “nhảy” qua nhiều tầng hàm rồi mới bị bắt ở đâu đó xa lắc — dễ bị nuốt mất hoặc xử lý sai chỗ. Với Go, ngay tại dòng gọi bạn *b buộc* phải quyết định: xử lý, hay trả lên trên. Không có đường lỗi “vô hình”.
- **Tool cần fail rõ ràng và có thông tin.** Một tool deploy nếu lỗi thì phải nói *cái gì* lỗi, *ở đâu*, để người vận hành xử lý. Cú pháp `%w` cho phép “bọc” lỗi kèm ngữ cảnh, tạo thành một chuỗi lỗi truy vết được.
- **Đọc code là thấy hết đường lỗi.** Khi review một tool hạ tầng, việc mọi điểm có thể hỏng đều hiện ra ngay trên màn hình (thay vì ẩn trong exception) khiến bạn khó bỏ sót edge case.

Vì sao Go chọn “dài dòng” thay vì “gọn gàng”? Vì với phần mềm hạ tầng, *chi phí của một lỗi bị nuốt* lớn hơn nhiều chi phí gõ thêm vài dòng `if`. Sự tương minh đổi lấy độ tin cậy — một cuộc mặc cả rất đáng với tool chạy trên production.

Concurrency với goroutine — vì sao hợp với tool gọi nhiều I/O

Tưởng tượng tool của bạn cần health-check 100 URL. Làm tuần tự, mỗi URL chờ 200ms, tổng cộng là 20 giây — dù CPU gần như ngồi chơi, chỉ *chờ mạng*. Đây chính là bài toán “I/O-bound” điển hình của DevOps tool.

Goroutine giải bài này rất gọn. Một goroutine là “luồng nhẹ” do runtime Go quản lý — tạo ra hàng nghìn cái tốn rất ít bộ nhớ. Bạn phóng 100 request đi *cùng lúc*, tổng thời gian rút xuống còn ~thời gian của request chậm nhất.

Vì sao goroutine hợp với I/O? Khi một goroutine “chờ” mạng, runtime Go tự động nhường CPU cho goroutine khác chạy — bạn không phải quản lý thread thủ công hay viết callback lồng nhau. Mô hình “phóng nhiều việc rồi chờ tất cả xong” khớp hoàn hảo với tool phải nói chuyện với nhiều service cùng lúc.

Ta sẽ dùng chính ý tưởng này trong CLI ngay dưới đây.

Viết CLI: flag (chuẩn) vs cobra (khi nào dùng)

Đây là tool health-check hoàn chỉnh, chạy được. Nó nhận danh sách URL, gọi song song bằng goroutine, và in trạng thái. Chỉ dùng thư viện chuẩn — không cần cài gì thêm.

```

package main

import (
    "flag"
    "fmt"
    "net/http"
    "os"
    "strings"
    "sync"
    "time"
)

// Result gói kết quả kiểm tra của một URL.
// Dùng struct để dữ liệu có tên rõ ràng, để mở rộng sau này.
type Result struct {
    URL      string
    Status   int
    Elapsed  time.Duration
    Err      error
}

// check gọi 1 URL và trả về Result. Không panic - mọi lỗi
// đều được đóng gói vào Result để caller tự quyết định.
func check(client *http.Client, url string) Result {
    start := time.Now()
    resp, err := client.Get(url)
    elapsed := time.Since(start)
    if err != nil {
        return Result{URL: url, Err: fmt.Errorf("request lỗi: %w", err), Elapsed:
elapsed}
    }
    defer resp.Body.Close()
    return Result{URL: url, Status: resp.StatusCode, Elapsed: elapsed}
}

func main() {
    // flag: khai báo cờ dòng lệnh chuẩn.
    urlList := flag.String("urls", "", "Danh sách URL, phân cách bằng dấu phẩy")
    timeout := flag.Duration("timeout", 5*time.Second, "Timeout mỗi request")
    flag.Parse()

    if *urlList == "" {
        fmt.Fprintln(os.Stderr, "Thiếu -urls. Ví dụ: healthcheck -
urls=https://a.com,https://b.com")
        os.Exit(2)
    }

    urls := strings.Split(*urlList, ",")
    client := &http.Client{Timeout: *timeout}

    // Gọi song song: mỗi URL một goroutine, WaitGroup để chờ tất cả xong.
    var wg sync.WaitGroup
    results := make([]Result, len(urls))
    for i, u := range urls {
        wg.Add(1)
        go func(idx int, url string) {

```

```

        defer wg.Done()
        results[idx] = check(client, strings.TrimSpace(url))
    }(i, u)
}
wg.Wait()

// In kết quả và đặt exit code để CI dùng được.
failed := 0
for _, r := range results {
    if r.Err != nil {
        fmt.Printf("[LỖI ] %s → %v (%s)\n", r.URL, r.Err,
r.Elapsed.Round(time.Millisecond))
        failed++
    } else if r.Status ≥ 200 && r.Status < 400 {
        fmt.Printf("[OK   ] %s → %d (%s)\n", r.URL, r.Status,
r.Elapsed.Round(time.Millisecond))
    } else {
        fmt.Printf("[XẤU ] %s → %d (%s)\n", r.URL, r.Status,
r.Elapsed.Round(time.Millisecond))
        failed++
    }
}

if failed > 0 {
    os.Exit(1) // exit code khác 0 để pipeline CI báo fail
}
}

```

Vài điểm đáng chú ý vì sao:

- **sync.WaitGroup** : cách kinh điển để “chờ N goroutine cùng hoàn thành”. `wg.Add(1)` trước khi phóng, `defer wg.Done()` khi xong, `wg.Wait()` để chặn `main` lại cho đến khi tất cả kết thúc. Vì sao cần? Nếu `main` thoát trước, các goroutine bị giết giữa chừng.
- **Mỗi goroutine ghi vào `results[idx]` riêng biệt**: không hai goroutine nào ghi cùng ô, nên không cần khóa (mutex). Đây là cách tránh race condition một cách đơn giản.
- **`os.Exit(1)` khi có lỗi**: tool DevOps *phải* trả exit code đúng để CI/CD biết bước này pass hay fail.

Với một tool đơn giản như trên, package `flag` là đủ và không cần thư viện ngoài. Nhưng khi tool lớn lên và cần **subcommand** (kiểu `git commit`, `kubectl get pods`), `flag` bắt đầu đuối. Lúc đó người ta chuyển sang `cobra` — chính là thư viện đứng sau `kubectl`, `helm`, `hugo`.

Tiêu chí	flag (thư viện chuẩn)	cobra (bên thứ ba)
Cần cài thêm	Không, có sẵn	Có (go get)
Subcommand (tool sub ...)	Phải tự xử lý thủ công	Hỗ trợ sẵn, cây lệnh lồng nhau
Help tự sinh	Cơ bản	Đẹp, tự sinh --help cho từng lệnh
Auto-complete shell	Không	Có (bash/zsh/fish)
Phù hợp khi	Tool 1 lệnh, script nhỏ	CLI nhiều lệnh, sản phẩm hoàn chỉnh

Vì sao đừng vội dùng cobra? Nguyên tắc “chọn công cụ nhỏ nhất đủ dùng”. Một tool một lệnh mà kéo cobra vào là thêm dependency, thêm boilerplate không cần thiết. Chỉ khi bạn thật sự cần cây subcommand thì cobra mới trả lại giá trị. Bắt đầu bằng **flag**, nâng cấp khi có nhu cầu thật.

Build & phân phối đa nền tảng (GOOS/GOARCH)

Khởi tạo module và build phiên bản cho chính máy bạn:

```
# Khởi tạo module (tạo file go.mod quản lý phụ thuộc)
go mod init github.com/ban/healthcheck

# Build ra binary cho HĐH hiện tại
go build -o healthcheck .

# Chạy thử
./healthcheck -urls=https://example.com,https://httpstat.us/500
```

Và đây là phần “siêu năng lực”: build cho các HĐH/kiến trúc khác **ngay trên máy hiện tại**, chỉ bằng hai biến môi trường **GOOS** (hệ điều hành) và **GOARCH** (kiến trúc CPU):

```
# Build cho Linux 64-bit (server, Docker image, CI runner)
GOOS=linux GOARCH=amd64 go build -o dist/healthcheck-linux-amd64 .

# Build cho Windows 64-bit
GOOS=windows GOARCH=amd64 go build -o dist/healthcheck-windows-amd64.exe .

# Build cho macOS Apple Silicon (M1/M2/M3)
GOOS=darwin GOARCH=arm64 go build -o dist/healthcheck-darwin-arm64 .
```

Vì sao đây là “siêu năng lực”? Với đa số ngôn ngữ, muốn có binary Windows bạn phải build *trên* Windows. Go tách rời máy build khỏi máy đích: một lệnh, một máy, ra binary cho mọi nền tảng. Đây chính là lý do một pipeline CI của Go có thể phát hành đồng thời hàng chục binary cho Linux/macOS/Windows x amd64/arm64 chỉ trong vài giây — và tại sao trang release trên GitHub của các Go tool luôn có sẵn file cho mọi HĐH.

Ứng dụng AI: generate boilerplate rồi tự bổ sung logic

Phần lặp đi lặp lại của một CLI Go — khai báo flag, dựng struct, khung `main`, vòng lặp goroutine + `WaitGroup` — là thứ AI viết rất nhanh và khá chuẩn. Cách làm hiệu quả:

1. **Mô tả rõ ý định cho AI:** “Viết CLI Go dùng package `flag`, nhận `-urls` phân cách bằng dấu phẩy, gọi song song bằng goroutine, in trạng thái và đặt exit code khác 0 khi có URL fail.” AI sẽ dựng cho bạn bộ khung gần giống đoạn code ở trên.
2. **Bạn bổ sung phần logic nghiệp vụ thật** — cái mà AI không biết: retry mấy lần? URL nào là “quan trọng” phải fail cả pipeline? định dạng output cần JSON để tool khác đọc?

Nhưng đừng dán code AI vào rồi chạy mù. Cần **review** kỹ những điểm AI hay sai:

- **Xử lý error:** AI đôi khi bỏ qua `err`, hoặc `log.Fatal` giữa chừng làm chết cả tool thay vì gom lỗi lại. Kiểm tra mọi `err` có được xử lý đúng không.
- **Đóng tài nguyên:** thiếu `defer resp.Body.Close()` gây rò rỉ kết nối — lỗi rất hay gặp.
- **Race condition:** nếu nhiều goroutine cùng ghi vào một biến/ `map` chung mà không có khóa, code chạy “có vẻ ổn” nhưng sẽ hỏng ngẫu nhiên. Chạy `go run -race` để kiểm tra.
- **Edge case:** chuỗi rỗng, URL thiếu `http://`, timeout, danh sách chỉ có dấu phẩy... AI thường bỏ sót.
- **Exit code:** tool có trả code đúng để CI dùng được không?

Vì sao vẫn phải review dù AI viết đúng cú pháp? Vì “biên dịch được” không có nghĩa là “đúng nghiệp vụ”. AI giỏi phần *hình thức* (boilerplate) nhưng không biết *ngữ cảnh vận hành* của bạn. Hãy để AI lo phần cơ khí, còn bạn giữ quyền quyết định về hành vi khi lỗi — nơi tool hạ tầng thắng hay thua.

Tóm tắt nhanh

- Go thống trị DevOps tooling vì trúng bài toán của lĩnh vực này: **một binary tĩnh** dễ phân phối, **cross-compile** dễ, **goroutine** hợp với I/O nhiều, hiệu năng cao, không cần runtime.
- Binary tĩnh đời độ phức tạp phụ thuộc từ máy người dùng sang thời điểm build — tool “chỉ một file” là tool ít rắc rối.
- Error kiểu Go tường minh (`if err != nil` , `%w`) đổi sự dài dòng lấy độ tin cậy — đáng giá cho phần mềm hạ tầng.
- Goroutine + `sync.WaitGroup` cho phép gọi song song hàng loạt I/O một cách gọn gàng.
- Dùng `flag` cho tool nhỏ; nâng lên `cobra` khi cần subcommand.
- `GOOS` / `GOARCH` cho phép build cho mọi nền tảng từ một máy — siêu năng lực phân phối.

Về nhà

Mở rộng tool `healthcheck` ở trên: thêm **một subcommand mới** tên `watch` để chạy health-check *lặp lại* theo chu kỳ.

Gợi ý:

- Vì `flag` không hỗ trợ subcommand sẵn, hãy đọc `os.Args[1]` để phân nhánh: `healthcheck check ...` (chạy một lần như hiện tại) và `healthcheck watch -interval=30s -urls= ...` (lặp lại).
- Dùng `flag.NewFlagSet("watch", ...)` để tạo bộ cờ riêng cho từng subcommand.
- Trong `watch`, dùng `time.NewTicker(interval)` để chạy lại mỗi chu kỳ; in kèm timestamp mỗi vòng.
- **Câu hỏi để tự trả lời:** đến khi nào thì việc tự xử lý subcommand bằng `os.Args` trở nên rối tới mức bạn *nên* chuyển sang cobra? (Thử tưởng tượng khi có 5 subcommand, mỗi cái vài flag riêng.)

Nâng cao (tùy chọn): thêm cờ `-json` để in kết quả dạng JSON (dùng `encoding/json`), giúp tool khác đọc được output.

Tự kiểm tra (câu hỏi “vì sao”)

1. **Vì sao** một binary tĩnh giúp việc phân phối tool dễ hơn hẳn so với một script Python? Điều gì được dời từ “máy người dùng” sang “máy build”?
2. **Vì sao** Go chọn trả về `error` tường minh thay vì dùng exception? Điều này giúp gì cho phần mềm hạ tầng chạy trên production?

3. **Vì sao** goroutine đặc biệt hợp với tool gọi nhiều I/O (như health-check nhiều URL), trong khi lợi ích của nó với tác vụ nặng CPU thì hạn chế hơn?
4. **Vì sao** không nên vội dùng cobra cho một tool chỉ có một lệnh? Khi nào thì cobra bắt đầu “trả lại giá trị”?
5. **Vì sao** trong ví dụ health-check, việc mỗi goroutine ghi vào `results[idx]` riêng giúp ta *không* cần dùng mutex?
6. **Vì sao** khi để AI generate CLI Go, bạn vẫn phải tự review phần error handling và edge case dù code đã biên dịch được?

PHẦN II

Bài giải lab

Lab 1

Bài giải Lab 1 — Dựng Jenkins & Freestyle job

Mục tiêu bài lab

Sau bài lab này bạn sẽ:

- Dựng được một server Jenkins chạy trong Docker, dữ liệu được lưu bền vững kể cả khi container bị xoá.
- Hoàn tất bước “unlock” ban đầu và cài các plugin cốt lõi (Git, Pipeline).
- Tạo được credentials để Jenkins truy cập GitHub một cách an toàn.
- Tạo và chạy một Freestyle job: kéo code từ repo về rồi chạy một lệnh test, sau đó đọc console output để hiểu Jenkins đã làm gì.

Mục tiêu thực sự không phải là “chạy cho xong” mà là hiểu **vì sao** mỗi bước tồn tại. Khi hiểu lý do, bạn sẽ tự sửa được lỗi khi cấu hình thực tế khác với hướng dẫn.

Bước 1 — Chạy Jenkins bằng Docker

Chúng ta dùng image chính thức `jenkins/jenkins:lts`. Bản `lts` (Long-Term Support) ổn định hơn bản `latest`, phù hợp cho môi trường học và production nhỏ.

```
docker run -d \  
  -p 8080:8080 \  
  -p 50000:50000 \  
  -v jenkins_home:/var/jenkins_home \  
  --name jenkins \  
  jenkins/jenkins:lts
```

Giải thích **vì sao** từng tham số:

- `-d` : chạy nền (detached), trả lại terminal cho bạn.
- `-p 8080:8080` : port giao diện web. Bạn truy cập Jenkins qua `http://localhost:8080`.
- `-p 50000:50000` : đây là port cho **agent (build node)** kết nối vào master qua giao thức JNLP. Bài này chưa dùng agent riêng, nhưng mở sẵn để sau này scale-out không phải tạo lại container.

- `-v jenkins_home:/var/jenkins_home` : **quan trọng nhất**. Toàn bộ cấu hình, job, plugin, credentials của Jenkins nằm trong thư mục `/var/jenkins_home` . Nếu không mount volume, khi container bị xoá thì mất sạch. Mount vào named volume `jenkins_home` giúp dữ liệu tồn tại độc lập với vòng đời container.
- `--name jenkins` : đặt tên cố định để các lệnh sau gọn gàng.

Kiểm tra container đã chạy:

```
docker ps
docker logs -f jenkins
```

Lần đầu khởi động Jenkins mất khoảng 30–60 giây để giải nén và khởi tạo. Nếu mở web quá sớm sẽ thấy trang “Please wait while Jenkins is getting ready”.

Bước 2 — Unlock & cài plugin

Mở `http://localhost:8080` , Jenkins yêu cầu nhập **initial admin password**. Đây là cơ chế bảo mật: chỉ người có quyền truy cập vào server (đọc được file bên trong container) mới unlock được lần đầu, tránh người lạ chiếm server ngay khi vừa dựng.

Lấy password bằng cách đọc file trong container:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Sao chép chuỗi in ra, dán vào ô “Administrator password”, bấm **Continue**.

Tiếp theo Jenkins hỏi cài plugin. Chọn **Install suggested plugins** — bộ này đã bao gồm **Git** và **Pipeline**, đúng thứ ta cần.

- **Git plugin**: để Jenkins biết cách `git clone` / `git fetch` từ repo. Không có nó, ô “Source Code Management” sẽ không có lựa chọn Git.
- **Pipeline plugin**: để sau này viết `Jenkinsfile` (khai báo pipeline bằng code). Bài này dùng Freestyle nhưng cứ cài sẵn cho các lab sau.

Sau khi plugin cài xong, tạo tài khoản admin đầu tiên (username/password của riêng bạn) rồi bấm **Save and Finish**.

Nếu lỡ bỏ qua bước chọn plugin, bạn vẫn có thể vào **Manage Jenkins** → **Plugins** → **Available** để cài Git và Pipeline sau, rồi restart Jenkins.

Bước 3 — Tạo credentials cho GitHub

Jenkins cần “danh tính” để truy cập GitHub. Với repo public thì clone không cần đăng nhập, nhưng ta vẫn tạo credentials để làm quen và để dùng được với repo private.

Vào **Manage Jenkins** → **Credentials** → **System** → **Global credentials** → **Add Credentials**.

- **Kind:** chọn *Username and password*.
- **Username:** tên đăng nhập GitHub của bạn.
- **Password:** **KHÔNG dùng mật khẩu GitHub**. Hãy vào GitHub → Settings → Developer settings → Personal access tokens, tạo một **token** (quyền **repo**) rồi dán vào ô này.
- **ID:** đặt tên gợi nhớ, ví dụ **github-token**.

Vì sao dùng token thay vì mật khẩu? GitHub đã bỏ xác thực bằng mật khẩu cho thao tác Git từ 2021. Token còn an toàn hơn: bạn thu hồi được từng token riêng lẻ mà không phải đổi mật khẩu chính.

Credentials được Jenkins lưu mã hoá trong **jenkins_home**. Đừng bao giờ ghi token thẳng vào script hay commit lên repo.

Bước 4 — Tạo Freestyle job kết nối repo

Từ Dashboard bấm **New Item** → nhập tên (ví dụ **lab01-freestyle**) → chọn **Freestyle project** → **OK**.

Cấu hình các phần:

1. Source Code Management → Git

- Repository URL: dán URL repo mẫu, ví dụ **https://github.com/<user>/<repo>.git**.
- Credentials: chọn **github-token** vừa tạo.
- Branch Specifier: đổi thành ***/main** nếu repo dùng nhánh **main** (mặc định Jenkins ghi ***/master**).

Vì sao chọn đúng branch? Nếu để `*/master` mà repo chỉ có `main`, build sẽ báo lỗi "couldn't find any revision to build".

2. **Build Steps** → **Add build step** → **Execute shell** (trên Linux/Docker). Nhập một lệnh test đơn giản:

```
echo "=== Bat dau build ==="
ls -la
echo "Commit hien tai:"
git log -1 --oneline
echo "=== Chay test gia lap ==="
test -f README.md && echo "README ton tai: OK" || echo "Khong tim thay README"
```

Container Jenkins chạy Linux nên dùng **Execute shell** (`sh`). Nếu bạn cài Jenkins trực tiếp trên Windows thì phải chọn **Execute Windows batch command** (`bat`) và đổi cú pháp lệnh cho phù hợp.

Bấm **Save**.

Bước 5 — Chạy thử & đọc console output

Ở trang job, bấm **Build Now**. Một mục build (ví dụ `#1`) xuất hiện ở "Build History". Bấm vào `#1` → **Console Output**.

Console output kỳ vọng đại khái:

```
Started by user admin
Cloning the remote Git repository
Cloning repository https://github.com/<user>/<repo>.git
> git init ...
> git fetch ...
Checking out Revision a1b2c3d (refs/remotes/origin/main)
[lab01-freestyle] $ /bin/sh -xe /tmp/jenkins123.sh
+ echo === Bat dau build ===
=== Bat dau build ===
+ ls -la
total 20
-rw-r--r-- 1 jenkins jenkins 102 README.md
+ git log -1 --oneline
a1b2c3d Initial commit
+ echo README ton tai: OK
README ton tai: OK
Finished: SUCCESS
```

Cách đọc: những dòng bắt đầu bằng `+` là lệnh Jenkins đang thực thi (nhờ cờ `-x`), dòng ngay dưới là kết quả. Dòng cuối `Finished: SUCCESS` báo build thành công (mã thoát 0). Nếu bất kỳ lệnh nào trả mã lỗi khác 0, Jenkins dừng và ghi `Finished: FAILURE`.

Kiểm chứng kết quả

Bạn coi như đạt bài lab khi:

- `docker ps` thấy container `jenkins` ở trạng thái `Up`.
- Đăng nhập web bằng tài khoản admin tự tạo (không còn cần initial password).
- Trong **Manage Jenkins** → **Plugins** → **Installed** có Git và Pipeline.
- Job `lab01-freestyle` build ra kết quả **SUCCESS**, console output có log clone repo và log của các lệnh test.
- Thử `docker restart jenkins` rồi vào lại, job và cấu hình vẫn còn (chứng minh volume hoạt động).

Lỗi thường gặp & cách xử lý

Lỗi thường gặp	Nguyên nhân	Cách xử lý
"Invalid password" khi unlock	Copy thiếu ký tự hoặc dính khoảng trắng/xuống dòng khi dán password	Chạy lại <code>docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword</code> , copy nguyên chuỗi, dán cẩn thận
Ồ SCM không có lựa chọn "Git"	Chưa cài Git plugin (bỏ qua bước suggested plugins)	Vào Manage Jenkins → Plugins → Available, cài "Git plugin", restart Jenkins
"Authentication failed" / "could not read Username" khi clone	Credentials sai, hoặc dùng mật khẩu GitHub thay vì token	Tạo Personal Access Token quyền <code>repo</code> , cập nhật lại credentials và chọn đúng ID trong job
"couldn't find any revision to build"	Branch specifier là <code>*/master</code> nhưng repo dùng <code>main</code>	Sửa Branch Specifier thành <code>*/main</code>
Build step báo lỗi cú pháp lệnh trên Windows	Dùng "Execute shell" (<code>sh</code>) trên Jenkins cài native Windows	Đổi sang "Execute Windows batch command" (<code>bat</code>), thay <code>ls</code> bằng <code>dir</code> v.v.
Mất hết job sau khi <code>docker rm</code>	Không mount volume <code>jenkins_home</code>	Luôn chạy container kèm <code>-v jenkins_home:/var/jenkins_home</code>

Mở rộng (Về nhà): cấu hình webhook

Mục tiêu: đẩy code lên GitHub là Jenkins **tự động build**, không cần bấm "Build Now".

Cơ chế: GitHub gửi một HTTP POST (webhook) tới Jenkins mỗi khi có push; Jenkins nhận được thì kích hoạt build tương ứng.

Các bước:

- Trong job, vào **Configure** → **Build Triggers**, tick **GitHub hook trigger for GITScm polling**.
- Trên GitHub repo: **Settings** → **Webhooks** → **Add webhook**.
 - Payload URL: `http://<địa-chỉ-jenkins>/github-webhook/` (chú ý dấu `/` cuối).

- Content type: `application/json` .
- Event: chọn *Just the push event*.

Nếu Jenkins chạy ở `localhost` , GitHub trên Internet **không** gọi vào được. Hãy dùng công cụ tunnel như `ngrok` (`ngrok http 8080`) để lấy một URL public tạm thời, rồi dán URL đó vào Payload URL.

Kiểm chứng: sửa một dòng trong README, commit rồi push. Sau vài giây, Build History của job xuất hiện build mới với dòng "Started by GitHub push". Vào phần **Recent Deliveries** của webhook trên GitHub để xem GitHub báo mã `200` — nghĩa là Jenkins đã nhận và xử lý thành công.

Lab 2

Bài giải Lab 2 — Jenkinsfile Build → Test → Deploy

Mục tiêu bài lab

Sau bài lab này bạn sẽ:

- Viết được một **Jenkinsfile Declarative** đặt ngay trong repo, gồm ba stage cốt lõi **Build** → **Test** → **Deploy** cho một app mẫu.
- Chạy pipeline dạng **Multibranch** để Jenkins tự phát hiện nhánh và tự build.
- Đọc **log** từng stage và **lưu artifact** bằng **archiveArtifacts**.
- Biết cách dùng AI (Claude/Copilot) để **sinh nhanh** Jenkinsfile rồi tự review, sửa cho đúng môi trường thật.

Vì sao đặt Jenkinsfile trong repo? Vì đây là triết lý *Pipeline as Code*: quy trình build đi cùng mã nguồn, được version-control, review qua Pull Request và tái lập được trên mọi máy Jenkins. Cấu hình bấm tay trên UI dễ mất và không truy vết được.

Bước 1 — Chuẩn bị app mẫu (Node.js)

Ta chọn Node.js làm ví dụ chính vì vòng đời **install → test → build** rất rõ ràng và phổ biến. Tạo một app tối giản với **package.json**:

```
{
  "name": "demo-app",
  "version": "1.0.0",
  "scripts": {
    "test": "node --test",
    "build": "mkdir -p dist && cp -r src dist/ && echo 'built' > dist/build.txt"
  }
}
```

Các lệnh chạy thử ở máy local trước khi đưa lên Jenkins:

```
npm ci      # cài dependency đúng theo package-lock.json (tái lập chính xác)
npm test    # chạy test, trả exit code ≠ 0 nếu fail
npm run build # tạo thư mục dist/ chứa sản phẩm build
```

Vì sao dùng `npm ci` thay cho `npm install`? `npm ci` cài **đúng** phiên bản đã khoá trong `package-lock.json`, xoá sạch `node_modules` cũ, nhanh và tái lập được — rất hợp cho môi trường CI. `npm install` có thể tự nâng version phụ và gây “chạy được ở máy tôi”.

Phương án thay thế: nếu dùng **Python**, thay bằng `pip install -r requirements.txt` → `pytest` → đóng gói `python -m build`. Nếu là **static site**, chỉ cần `npm run build` sinh thư mục `dist/` là đủ, có thể bỏ stage Test hoặc thay bằng kiểm tra HTML lint.

Bước 2 — Viết Jenkinsfile (Declarative) Build → Test → Deploy

Đặt file tên đúng là `Jenkinsfile` ở gốc repo:

```

pipeline {
    agent any

    options {
        timestamps()
        timeout(time: 15, unit: 'MINUTES')
    }

    environment {
        DEPLOY_DIR = "/var/www/demo-app"
        NODE_ENV   = "production"
    }

    stages {
        stage('Build') {
            steps {
                echo "Đang build trên nhánh ${env.BRANCH_NAME}"
                sh 'npm ci'
                sh 'npm run build'
            }
        }

        stage('Test') {
            steps {
                sh 'npm test'
            }
        }

        stage('Deploy') {
            when {
                branch 'main'
            }
            steps {
                echo "Deploy sang ${DEPLOY_DIR}"
                sh '''
                    mkdir -p "$DEPLOY_DIR"
                    cp -r dist/* "$DEPLOY_DIR"/
                '''
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'dist/**', allowEmptyArchive: true, fingerprint:
true
        }
        success {
            echo "Pipeline THÀNH CÔNG trên nhánh ${env.BRANCH_NAME}"
        }
        failure {
            echo "Pipeline THẤT BẠI - kiểm tra log stage vừa lỗi"
        }
    }
}

```

Giải thích từng phần **vì sao**:

- **agent any** : cho Jenkins chạy pipeline trên bất kỳ node/agent nào rảnh. Ở dự án thật bạn có thể ghim `agent { label 'linux' }` để chọn máy có đúng công cụ.
- **options** : `timestamps()` thêm dấu thời gian vào từng dòng log (để tính thời gian mỗi bước); `timeout` chặn pipeline treo vô hạn khi lệnh nào đó bị kẹt.
- **environment** : khai báo biến dùng lại nhiều nơi. Tách cấu hình ra khỏi lệnh giúp đổi đường dẫn deploy mà không phải sửa từng stage.
- **stage('Build')** : `npm ci` rồi `npm run build` . Đặt Build trước để nếu code không cài/không build được thì dừng sớm, khởi phí thời gian test.
- **stage('Test')** : `npm test` trả exit code khác 0 khi có test fail → Jenkins tự đánh dấu stage đỏ và dừng pipeline. Đây là "cổng chất lượng".
- **stage('Deploy')** với `when { branch 'main' }` : chỉ deploy khi build **nhánh main**. Vì sao? Ta không muốn mỗi nhánh feature đều ghi đè server production; các nhánh khác vẫn build + test bình thường nhưng bỏ qua deploy.
- **post** : khối chạy **sau** stages. `always` luôn lưu artifact dù thành/bại; `success` / `failure` để phản ứng theo kết quả (bước Về nhà sẽ gắn thông báo vào đây).
- **archiveArtifacts** với `allowEmptyArchive: true` tránh làm hỏng build khi `dist/` trống; `fingerprint: true` để truy vết chính xác bản build nào đã tạo file nào.

Bước 3 — Tạo Multibranch Pipeline trong Jenkins

1. Đẩy repo (kèm `Jenkinsfile`) lên Git (GitHub/GitLab/Gitea).
2. Trên Jenkins: **New Item** → **Multibranch Pipeline** → đặt tên.
3. Mục **Branch Sources** → Add source **Git/GitHub**, dán URL repo và chọn credential nếu repo private.
4. **Build Configuration**: để **by Jenkinsfile**, `Script Path` = `Jenkinsfile` .
5. Lưu. Jenkins chạy **Scan Repository**: quét mọi nhánh có `Jenkinsfile` và tạo một job con cho mỗi nhánh.

Vì sao chọn Multibranch thay vì Pipeline thường? Multibranch **tự phát hiện** nhánh mới và Pull Request rồi tự tạo pipeline tương ứng — không phải cấu hình tay từng nhánh. Rất hợp quy trình nhiều nhánh feature.

Muốn tự chạy khi có commit: bật **Scan Repository Triggers** → **Periodically** (ví dụ mỗi 1 phút) hoặc cấu hình **webhook** từ GitHub để Jenkins build tức thì.

Bước 4 — Xem log & lưu artifact

- Vào job con của một nhánh → chọn số build (ví dụ `#1`) → **Console Output** để xem log đầy đủ.
- **Stage View** / **Blue Ocean** hiển thị trực quan từng stage xanh/đỏ và thời gian chạy.
- Sản phẩm build được liệt kê ở trang build dưới mục **Last Successful Artifacts**, tải về từ đường dẫn `dist/**` mà `archiveArtifacts` đã lưu.

Log là công cụ debug số một: khi một stage đỏ, mở Console Output và kéo tới đúng lệnh `sh` gây lỗi, đọc dòng exit code và thông báo ngay phía trên.

Kiểm chứng kết quả

Pipeline coi như đạt khi:

- Nhánh `main`: cả ba stage **Build** → **Test** → **Deploy** đều xanh.
- Nhánh feature: **Build** → **Test** xanh, stage **Deploy** hiện *skipped* (do điều kiện `when`).
- Trang build có artifact `dist/**` tải về được.
- Cố tình làm một test fail → pipeline chuyển đỏ ngay tại stage Test và **không** chạy Deploy. Điều này chứng minh cổng chất lượng hoạt động.

Ứng dụng AI: sinh nhanh Jenkinsfile

Bạn có thể mô tả nhu cầu cho Claude/Copilot để nhận bản nháp Jenkinsfile trong vài giây, sau đó review lại. Một **prompt** tốt cần nêu đủ ngữ cảnh:

```
Viết một Jenkinsfile Declarative cho app Node.js 20.
Yêu cầu:
- agent any, dùng npm ci để cài dependency.
- 3 stage: Build (npm run build), Test (npm test), Deploy chỉ chạy khi nhánh là main.
- Deploy copy thư mục dist/ sang /var/www/demo-app.
- post: luôn archiveArtifacts 'dist/**'; gửi thông báo khi failure.
- Thêm timeout 15 phút và timestamps.
Giải thích ngắn từng stage.
```

Checklist cần review sau khi AI sinh code (đừng chạy mù):

- ☐ **Version Node** đúng với dự án chưa (dùng `tools` / `nvm` / Docker image nào)?
- ☐ **Đường dẫn deploy** có đúng server thật không, có ghi đè nhầm production không?

- ☐ **Secret/credential** có bị hardcode token, mật khẩu trong file không? Phải dùng `credentials()` / `withCredentials`.
- ☐ Điều kiện `when { branch 'main' }` đã khớp tên nhánh chính (main vs master) chưa?
- ☐ Lệnh `sh` là cú pháp Linux — agent có phải Windows (`bat`) không?
- ☐ `archiveArtifacts` trả đúng thư mục sản phẩm chưa.

AI giúp bạn *khởi động nhanh*, nhưng bạn chịu trách nhiệm cuối cùng. Luôn đọc kỹ phần deploy và secret trước khi merge.

Lỗi thường gặp & cách xử lý

Lỗi thường gặp	Nguyên nhân	Cách xử lý
<code>npm: command not found</code>	Agent chưa cài Node/npm hoặc không có trong PATH	Cài NodeJS plugin và dùng khối <code>tools { nodejs 'Node20' }</code> , hoặc chạy trên agent có Node
Pipeline không tự chạy khi push	Chưa bật webhook / scan trigger	Bật Scan Repository Triggers hoặc cấu hình webhook Git; bấm Scan Repository Now để test
Stage Deploy chạy trên mọi nhánh	Thiếu điều kiện <code>when</code>	Thêm <code>when { branch 'main' }</code> vào stage Deploy
<code>archiveArtifacts</code> báo không tìm thấy file	Build chưa tạo <code>dist/</code> hoặc sai đường dẫn glob	Kiểm tra stage Build tạo đúng thư mục; đặt <code>allowEmptyArchive: true</code> khi hợp lý
Permission denied khi copy sang <code>/var/www</code>	User chạy Jenkins không có quyền ghi	Cấp quyền thư mục, hoặc deploy qua <code>sudo</code> / <code>ssh</code> /rsync với credential riêng

Mở rộng (Về nhà): thông báo khi build fail

Nhiệm vụ về nhà: thêm thông báo tự động khi pipeline **fail** để đội biết ngay. Gắn vào khối `post { failure { ... } }`.

Gửi **email** (cần cấu hình SMTP ở *Manage Jenkins* → *System*, dùng plugin **Email Extension** hoặc Mailer có sẵn):

```

post {
  failure {
    mail to: 'team@company.com',
        subject: "BUILD FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        body: "Nhánh ${env.BRANCH_NAME} build lỗi. Xem log: ${env.BUILD_URL}"
    }
  }
}

```

Gửi **Slack** (cần cài plugin **Slack Notification** và cấu hình token workspace):

```

post {
  failure {
    slackSend channel: '#ci-alerts',
              color: 'danger',
              message: "Build lỗi: ${env.JOB_NAME} #${env.BUILD_NUMBER}
(${env.BUILD_URL})"
    }
  }
}

```

Vì sao đặt trong **failure** mà không phải **always** ? Để tránh spam — chỉ báo khi có sự cố thật. Có thể thêm **fixed { ... }** để báo "đã xanh trở lại" sau khi sửa xong. Nhớ dùng credential Jenkins cho token Slack/SMTP, tuyệt đối không hardcode.

Lab 3

Bài giải Lab 3 — Ansible playbook cài Nginx & tích hợp Jenkins

Mục tiêu bài lab

Sau bài lab này, bạn sẽ hiểu và làm được:

- Khai báo **inventory** để Ansible biết cần cấu hình những máy nào.
- Viết một **playbook** cài đặt và cấu hình Nginx trên node target bằng các module chuẩn (thay vì gõ lệnh thủ công).
- Hiểu khái niệm **idempotency** — chạy playbook nhiều lần vẫn cho cùng một kết quả cuối, lần thứ hai không thay đổi gì (`changed=0`).
- Gọi playbook từ **Jenkins pipeline** để tự động hóa việc deploy.
- Ứng dụng AI để sinh **inventory đa môi trường** (dev/staging/prod) với biến riêng cho từng môi trường.
- Đóng gói playbook thành **Role** tái sử dụng.

Triết lý cốt lõi của Ansible là *khai báo trạng thái mong muốn* (desired state), không phải *ra lệnh từng bước*. Bạn mô tả "Nginx phải được cài và đang chạy", Ansible tự lo phần còn lại.

Bước 1 — Chuẩn bị SSH & inventory

Ansible điều khiển máy target qua SSH, **không cần cài agent** trên target (agentless). Điều kiện tiên quyết là máy control (nơi chạy Ansible) SSH được vào target không cần mật khẩu.

Tạo và copy SSH key:

```
ssh-keygen -t ed25519 -f ~/.ssh/ansible_key -N ""  
ssh-copy-id -i ~/.ssh/ansible_key.pub deploy@192.168.56.20
```

Tạo file **inventory** (định dạng INI) khai báo nhóm host:

```
[web]
web1 ansible_host=192.168.56.20

[web:vars]
ansible_user=deploy
ansible_ssh_private_key_file=~/.ssh/ansible_key
ansible_python_interpreter=/usr/bin/python3
```

Giải thích **vì sao**:

- **[web]** là một **nhóm** — sau này playbook chỉ cần nhắm vào **hosts: web** là áp dụng cho mọi máy trong nhóm. Gom nhóm giúp mở rộng từ 1 lên 100 máy mà không sửa playbook.
- **[web:vars]** đặt biến chung cho cả nhóm: user đăng nhập, key SSH, và đường dẫn Python. Khai báo **ansible_python_interpreter** tránh lỗi khi target có nhiều bản Python.

Kiểm tra kết nối trước khi làm gì thêm:

```
ansible -i inventory web -m ping
```

Nếu trả về **"ping": "pong"** là SSH và Python trên target đã sẵn sàng.

Bước 2 — Viết playbook cài Nginx

Tạo file **templates/nginx.conf.j2** (một site tối giản):

```
server {
    listen 80 default_server;
    server_name {{ server_name }};
    root /var/www/html;
    index index.html;
}
```

Tạo playbook **site.yml** :

```

---
- name: Cài đặt và cấu hình Nginx
  hosts: web
  become: true
  vars:
    server_name: "example.local"

  tasks:
    - name: Cài gói nginx
      ansible.builtin.package:
        name: nginx
        state: present

    - name: Copy file cấu hình từ template
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/conf.d/default.conf
        owner: root
        group: root
        mode: "0644"
      notify: Restart nginx

    - name: Đảm bảo nginx đang chạy và bật khi khởi động
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted

```

Giải thích **vì sao dùng module thay vì `shell`** :

- Module `package` / `service` / `template` là **idempotent**: chúng kiểm tra trạng thái hiện tại trước khi hành động. Nếu Nginx đã cài rồi thì task báo **ok** chứ không cài lại. Ngược lại nếu bạn viết `shell: apt install -y nginx`, Ansible không biết trạng thái, luôn báo **changed**, và không an toàn khi chạy lặp.
- `package` là module trung lập — tự chọn `apt` trên Debian/Ubuntu hoặc `yum` / `dnf` trên RHEL/CentOS, nên playbook chạy được đa nền tảng.
- `template` render file Jinja2, thay biến `{{ server_name }}` bằng giá trị thực, đồng thời tự so sánh nội dung: chỉ ghi đè khi file đích khác nội dung mong muốn.

Giải thích **vì sao có handler**:

- Handler `Restart nginx` chỉ chạy **khi được notify** — tức là chỉ khi task template thực sự thay đổi file cấu hình. Nếu cấu hình không đổi, Nginx không bị restart vô ích, tránh gián đoạn dịch vụ.

- Handler chạy **một lần ở cuối** playbook dù được notify nhiều lần, nên nếu nhiều task cùng đưng vào cấu hình, Nginx chỉ restart đúng một lần.

Bước 3 — Chạy playbook & kiểm tra idempotency

Chạy lần đầu:

```
ansible-playbook -i inventory site.yml
```

Output mẫu **lần 1** (mọi thứ được tạo mới → **changed**):

```
PLAY [Cai dat va cau hinh Nginx] *****

TASK [Gathering Facts] *****
ok: [web1]

TASK [Cai goi nginx] *****
changed: [web1]

TASK [Copy file cau hinh tu template] *****
changed: [web1]

TASK [Dam bao nginx dang chay va bat khi khoi dong] *****
changed: [web1]

RUNNING HANDLER [Restart nginx] *****
changed: [web1]

PLAY RECAP *****
web1 : ok=5  changed=4  unreachable=0  failed=0  skipped=0
```

Chạy **lần 2** (không sửa gì) — trạng thái đã đúng nên tất cả báo **ok**, handler không chạy:

```
PLAY RECAP *****
web1 : ok=4  changed=0  unreachable=0  failed=0  skipped=0
```

changed=0 ở lần 2 chính là bằng chứng **idempotency**: playbook mô tả trạng thái, và khi trạng thái đã đạt, Ansible không làm lại. Đây là tiêu chí quan trọng để playbook an toàn chạy trong pipeline tự động.

Nếu lần 2 vẫn còn task `changed`, thường là do bạn dùng `command` / `shell` không có `creates:` / `changed_when:`, hoặc template render ra nội dung khác nhau mỗi lần (ví dụ có timestamp). Hãy soi lại task đó.

Bước 4 — Gọi playbook từ Jenkins pipeline

Đặt inventory, `site.yml` và `templates/` vào cùng repo với `Jenkinsfile`. Thêm stage sau vào pipeline:

```
stage('Deploy with Ansible') {
    steps {
        sshagent(credentials: ['ansible-ssh-key']) {
            sh '''
                ansible --version
                ansible-playbook -i inventory site.yml --diff | tee ansible.log
            '''
        }
    }
}
```

Giải thích **vì sao**:

- `sshagent` nạp SSH private key (lưu trong Jenkins Credentials, id `ansible-ssh-key`) vào agent để `ansible-playbook` SSH được vào target mà **không hard-code key trong repo**.
- `--diff` in ra khác biệt của file cấu hình khi có thay đổi — hữu ích để review trong log build.
- `tee ansible.log` lưu output để lưu vết (có thể `archiveArtifacts 'ansible.log'`).

Nên đặt `ANSIBLE_HOST_KEY_CHECKING=false` cho môi trường CI hoặc quản lý `known_hosts` sẵn, nếu không lần SSH đầu tiên trong CI sẽ treo chờ xác nhận fingerprint.

Kiểm chứng kết quả

Sau khi pipeline chạy xong, kiểm tra trên chính máy target hoặc từ máy control:

```
# Trạng thái dịch vụ
ansible -i inventory web -m shell -a "systemctl is-active nginx" --become

# Kiểm tra HTTP trả về 200
curl -I http://192.168.56.20
```

Kết quả mong đợi: `systemctl is-active` trả về `active`, và `curl -I` trả `HTTP/1.1 200 OK`.
 Kết hợp với `changed=0` ở lần chạy thứ hai, bạn đã chứng minh được playbook vừa **đúng** vừa **idempotent**.

Ứng dụng AI: inventory đa môi trường

Nhờ AI sinh nhanh cấu trúc inventory cho ba môi trường, mỗi môi trường có biến riêng. Gợi ý prompt:

“Tạo cho tôi cấu trúc thư mục Ansible với ba inventory dev/staging/prod, mỗi môi trường dùng group_vars riêng cho server_name và số worker của Nginx.”

Cấu trúc thư mục sinh ra:

```

ansible/
├── inventories/
│   ├── dev.ini
│   ├── staging.ini
│   └── prod.ini
├── group_vars/
│   ├── dev/
│   │   └── nginx.yml      # server_name: dev.example.local ; worker_processes: 1
│   ├── staging/
│   │   └── nginx.yml     # server_name: staging.example.local ; worker_processes: 2
│   └── prod/
│       └── nginx.yml     # server_name: example.com ; worker_processes: auto
├── templates/
│   └── nginx.conf.j2
└── site.yml
  
```

Ví dụ `group_vars/prod/nginx.yml`:

```

---
server_name: "example.com"
worker_processes: "auto"
  
```

Khi chạy, chỉ cần đổi inventory:

```
ansible-playbook -i inventories/prod.ini site.yml
```

Ansible tự nạp biến trong `group_vars/prod/` cho các host thuộc nhóm tương ứng. Cùng một `site.yml` nhưng ra ba cấu hình khác nhau — **tách bạch code và cấu hình**.

Luôn kiểm tra lại giá trị AI sinh ra, đặc biệt `server_name` prod và các thông tin nhạy cảm. AI dùng để tăng tốc, không thay thế việc review.

Lỗi thường gặp & cách xử lý

Lỗi thường gặp	Nguyên nhân	Cách xử lý
<code>Permission denied (publickey) / UNREACHABLE</code>	SSH key chưa được copy sang target hoặc sai đường dẫn key	Chạy <code>ssh-copy-id</code> , kiểm tra <code>ansible_ssh_private_key_file</code> , thử <code>ssh</code> tay trước
<code>Missing sudo password</code> khi <code>become: true</code>	User không có sudo NOPASSWD, thiếu quyền nâng đặc quyền	Thêm <code>--ask-become-pass</code> hoặc cấu hình NOPASSWD trong sudoers cho user deploy
<code>/usr/bin/python: not found</code>	Target không có Python ở đường dẫn Ansible tìm	Khai báo <code>ansible_python_interpreter=/usr/bin/python3</code> trong inventory
Host key verification failed (trong CI)	<code>known_hosts</code> chưa có fingerprint của target	Đặt <code>ANSIBLE_HOST_KEY_CHECKING=false</code> hoặc pre-populate <code>known_hosts</code>
Lần 2 vẫn <code>changed</code>	Dùng <code>shell</code> / <code>command</code> không idempotent	Chuyển sang module chuẩn, hoặc thêm <code>creates:</code> / <code>changed_when:</code>

Mở rộng (Về nhà): chuyển playbook thành Role

Khi playbook lớn dần, gom logic Nginx thành một **Role** giúp tái sử dụng và chia sẻ. Tạo bằng lệnh:

```
ansible-galaxy init roles/nginx
```

Cấu trúc Role chuẩn:

```

roles/
└─ nginx/
    └─ tasks/
        └─ main.yml          # các task cài đặt & cấu hình
    └─ handlers/
        └─ main.yml          # handler restart nginx
    └─ templates/
        └─ nginx.conf.j2     # file cấu hình Jinja2
    └─ defaults/
        └─ main.yml          # biến mặc định: server_name, worker_processes
    └─ vars/
        └─ main.yml          # biến có ưu tiên cao hơn
    └─ files/
        # file tính copy nguyên vẹn
    └─ meta/
        └─ main.yml          # metadata & dependencies

```

Sau khi tách, `site.yml` gọn lại chỉ còn “gọi role”:

```

---
- name: Deploy web tier
  hosts: web
  become: true
  roles:
    - nginx

```

Giải thích **vì sao dùng Role**:

- Ansible **tự nạp** `tasks/main.yml`, `handlers/main.yml`, `templates/` theo quy ước — bạn không cần khai báo đường dẫn thủ công.
- `defaults/main.yml` chứa biến mặc định (ưu tiên thấp), dễ dàng override bằng `group_vars` cho từng môi trường — kết hợp hoàn hảo với phần inventory đa môi trường ở trên.
- Role có thể đóng gói, đẩy lên Ansible Galaxy hoặc chia sẻ giữa nhiều dự án, giúp chuẩn hóa cách deploy trong cả team.

Nguyên tắc: một Role nên làm tốt **một việc** (ví dụ “cài Nginx”). Muốn deploy full stack thì viết nhiều role nhỏ rồi kết hợp trong một playbook, thay vì một role khổng lồ.

Lab 4

Bài giải Lab 4 — Terraform & Docker Compose (Config as Code)

Mục tiêu bài lab

Sau Lab 4 bạn sẽ hiểu tư duy **Config as Code (Infrastructure as Code)**: thay vì click chuột dựng hạ tầng bằng tay, ta *mô tả* hạ tầng mong muốn bằng file văn bản, đưa vào Git và để công cụ tự dựng. Cụ thể bạn sẽ:

- Dùng **Terraform** (hoặc OpenTofu) với provider `kreuzwerker/docker` để tạo một container nginx local.
- Hiểu vòng đời `init → plan → apply` và **vì sao** phải có bước `plan` cùng với **state**.
- Viết **docker-compose.yml** mô tả stack gồm `app` (Node) + `db` (Postgres) và chạy thử.
- Đưa toàn bộ config vào Git, thảo luận quy trình **review qua Pull Request** cho hạ tầng.
- Dùng AI để sinh khung nhanh và tự kiểm (checklist review).

Ý tưởng cốt lõi: hạ tầng trở thành *dữ liệu có thể review, version, rollback* — y hệt mã nguồn ứng dụng.

Bước 1 — Terraform tạo Docker container

Tạo thư mục làm việc và một file duy nhất `main.tf`. File này khai báo *trạng thái mong muốn* (desired state): "tôi muốn có một image nginx và một container chạy từ image đó".

```

terraform {
  required_version = "≥ 1.5.0"

  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "→ 3.0" # pin version để build tái lập được
    }
  }
}

# Provider là "cầu nối" giữa Terraform và Docker daemon local
provider "docker" {
  host = "npipe:////./pipe/docker_engine" # Windows; Linux/Mac: unix:///var/run/docker.sock
}

# Resource 1: kéo image nginx về máy
resource "docker_image" "nginx" {
  name          = "nginx:1.27-alpine"
  keep_locally = true # không xóa image khi destroy, tiết kiệm lần sau
}

# Resource 2: tạo container từ image ở trên
resource "docker_container" "web" {
  name = "lab04-nginx"
  image = docker_image.nginx.image_id # tham chiếu ⇒ Terraform tự suy ra thứ tự

  ports {
    internal = 80    # cổng bên trong container
    external = 8080 # cổng map ra host ⇒ http://localhost:8080
  }

  restart = "unless-stopped"
}

```

Giải thích từng phần:

- **terraform.required_providers** : khai báo lấy provider ở đâu và **pin version** (**→ 3.0**). Pin giúp lần chạy sau vẫn dùng đúng phiên bản, tránh "hôm nay chạy được mai vỡ".
- **provider "docker"** : nói cho Terraform biết kết nối tới Docker daemon nào. Trên Windows dùng named pipe, trên Linux/Mac dùng Unix socket.
- **docker_image.nginx** : một *resource* mô tả image cần có. Terraform sẽ pull nếu chưa có.
- **docker_container.web** : container thực tế. Nhờ tham chiếu **docker_image.nginx.image_id** , Terraform tự hiểu container **phụ thuộc** vào image và tạo image trước. Đây là *implicit dependency* — không cần khai báo thứ tự thủ công.

Bước 2 — terraform init / plan / apply (và vì sao có plan)

```
# 1) Tải provider về, khởi tạo thư mục .terraform
terraform init

# 2) Xem TRƯỚC Terraform định làm gì (không đổi gì thật)
terraform plan

# 3) Thực thi: tạo image + container thật
terraform apply
# gõ "yes" khi được hỏi xác nhận

# Kiểm tra
docker ps          # thấy container lab04-nginx
curl http://localhost:8080

# Dọn dẹp khi xong
terraform destroy
```

Vì sao **plan** quan trọng? **plan** so sánh 3 thứ: (1) cấu hình bạn viết, (2) **state** đang lưu, (3) hạ tầng thực tế — rồi in ra *diff*: cái gì sẽ **thêm (+)**, **đổi (~)**, **xóa (-)**. Đây là bước “xem trước hóa đơn” trước khi bấm mua: bạn phát hiện sớm việc apply có vô tình xóa database production hay không. Trên CI, **plan** thường được đính vào Pull Request để người review nhìn thấy tác động trước khi merge.

Vì sao có **state**? File **terraform.tfstate** là “sổ ghi nhớ” ánh xạ giữa resource trong code (**docker_container.web**) và ID thật ngoài đời (**a1b2c3 ...**). Không có state, Terraform không biết container nào nó từng tạo để mà cập nhật/xóa — nó sẽ tạo trùng. State cũng lưu thông tin nhạy cảm, nên team thật đặt state ở **remote backend** (S3, GCS...) có khóa (lock) để hai người không apply đè lên nhau.

Không bao giờ sửa tay **terraform.tfstate**. Và thêm ***.tfstate*** vào **.gitignore** vì nó có thể chứa secret.

Bước 3 — docker-compose.yml cho stack app + db

Terraform hợp để quản *hạ tầng*; còn để dựng nhanh một *stack ứng dụng nhiều container liên kết nhau* khi phát triển, **docker compose** gọn hơn. Tạo **docker-compose.yml** :

```

services:
  app:
    build: ./app          # dùng Dockerfile trong ./app
    ports:
      - "3000:3000"       # host:container
    environment:
      DATABASE_URL: "postgres://appuser:secret@db:5432/appdb"
      # "db" là tên service ⇒ Compose tự phân giải DNS nội bộ
    depends_on:
      db:
        condition: service_healthy # chỉ start app khi db đã sẵn sàng
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
    volumes:
      - db_data:/var/lib/postgresql/data # dữ liệu sống sót qua restart
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 5s
      timeout: 3s
      retries: 5

# Volume có tên: Docker quản lý, không mất khi "compose down"
volumes:
  db_data:

```

Giải thích từng phần:

- **services** : mỗi service là một container. Ở đây **app** và **db** nằm chung một mạng ảo mặc định nên **app** gọi **db** qua *tên service* thay vì IP.
- **depends_on** + **condition: service_healthy** : kiểm soát thứ tự khởi động. Không có nó, **app** có thể bật lên trước khi Postgres nhận kết nối và crash. **healthcheck** của **db** cho Compose biết khi nào db thực sự “khỏe”.
- **volumes: db_data** : đây là điểm mấu chốt về *bền vững dữ liệu*. Nếu không gắn volume, container **db** bị xóa là mất sạch data. Named volume tách vòng đời dữ liệu khỏi vòng đời container.
- **environment** : cấu hình truyền vào qua biến môi trường — nguyên tắc 12-factor. (Ở lab thì để mật khẩu thẳng cho dễ; thật thì dùng **.env** /secret manager.)

```

docker compose up -d      # dựng cả stack ở chế độ nền
docker compose ps         # xem trạng thái/health
docker compose logs -f app
docker compose down       # dừng & xóa container (volume vẫn còn)

```

Bước 4 — Đưa config vào Git & review qua Pull Request

Vì tất cả hạ tầng giờ chỉ là *file văn bản*, hãy quản chúng bằng Git y như code.

```
git init
# .gitignore
printf ".terraform/\n*.tfstate*\n.env\n" > .gitignore

git add main.tf docker-compose.yml .gitignore
git commit -m "lab04: terraform nginx + compose app/db stack"

git checkout -b feature/add-redis-cache
# ... sửa config ...
git commit -am "feat: thêm service redis vào stack"
git push origin feature/add-redis-cache
# Mở Pull Request trên GitHub/GitLab
```

Vì sao đưa config vào Git?

- **Versioned:** mọi thay đổi hạ tầng có lịch sử — ai đổi, đổi gì, khi nào (`git log` , `git blame`).
- **Review:** đồng đội đọc diff *trước khi* thay đổi chạm hệ thống thật, bắt lỗi sớm.
- **Rollback:** sự cố thì `git revert` về commit tốt rồi apply lại — quay lui trong vài phút thay vì dựng lại bằng tay.
- **Nguồn sự thật duy nhất:** hạ tầng khớp với repo, chống "config trôi" (config drift) do sửa tay.

Quy trình PR review hạ tầng khác review code thường ở chỗ *sai một dòng có thể sập production*: PR nên đính kèm output `terraform plan` để reviewer thấy trước diff; reviewer soi kỹ resource bị `-` (xóa), thay đổi biến môi trường/secret, và version của provider. Chỉ merge khi CI xanh và có ít nhất một approve.

Kiểm chứng kết quả

- `terraform apply` xong: `docker ps` thấy `lab04-nginx` , mở `http://localhost:8080` ra trang chào của nginx.
- `docker compose up -d` xong: `docker compose ps` báo cả `app` và `db` đều `running` / `healthy` ; `app` kết nối được Postgres (xem log không có lỗi `ECONNREFUSED`).
- `docker compose down` rồi `up` lại: dữ liệu trong `db` **vẫn còn** nhờ named volume — đây là bằng chứng volume hoạt động đúng.
- `git log --online` cho thấy lịch sử thay đổi config rõ ràng.

Ứng dụng AI: sinh khung Terraform/Compose

AI rất hợp để *sinh khung nháp* nhanh, còn bạn giữ vai trò rà soát. Ví dụ prompt:

"Sinh giúp `main.tf` dùng provider `kreuzwerker/docker` tạo 1 container nginx map cổng 8080, có pin provider version. Và một `docker-compose.yml` gồm service Node app + Postgres, có volume cho db, `depends_on` với healthcheck. Ghi chú ngăn từng khối."

Checklist review output của AI (đừng tin mù):

- ☐ **Provider version có được pin không?** (`~>` thay vì để trống → tránh vỡ build).
- ☐ **State:** đã bỏ `*.tfstate*` vào `.gitignore` chưa? Có vô tình commit state không?
- ☐ **Secret:** mật khẩu/API key có bị hardcode và đẩy lên Git không? Nên chuyển sang `.env` /secret manager.
- ☐ **Resource nguy hiểm:** `plan` có dòng `-` (xóa) nào ngoài dự kiến không?
- ☐ **Volume:** db có volume để không mất dữ liệu không?
- ☐ Cú pháp/kiểu tài nguyên có *thật sự tồn tại* trong provider không (AI hay bịa tên field).

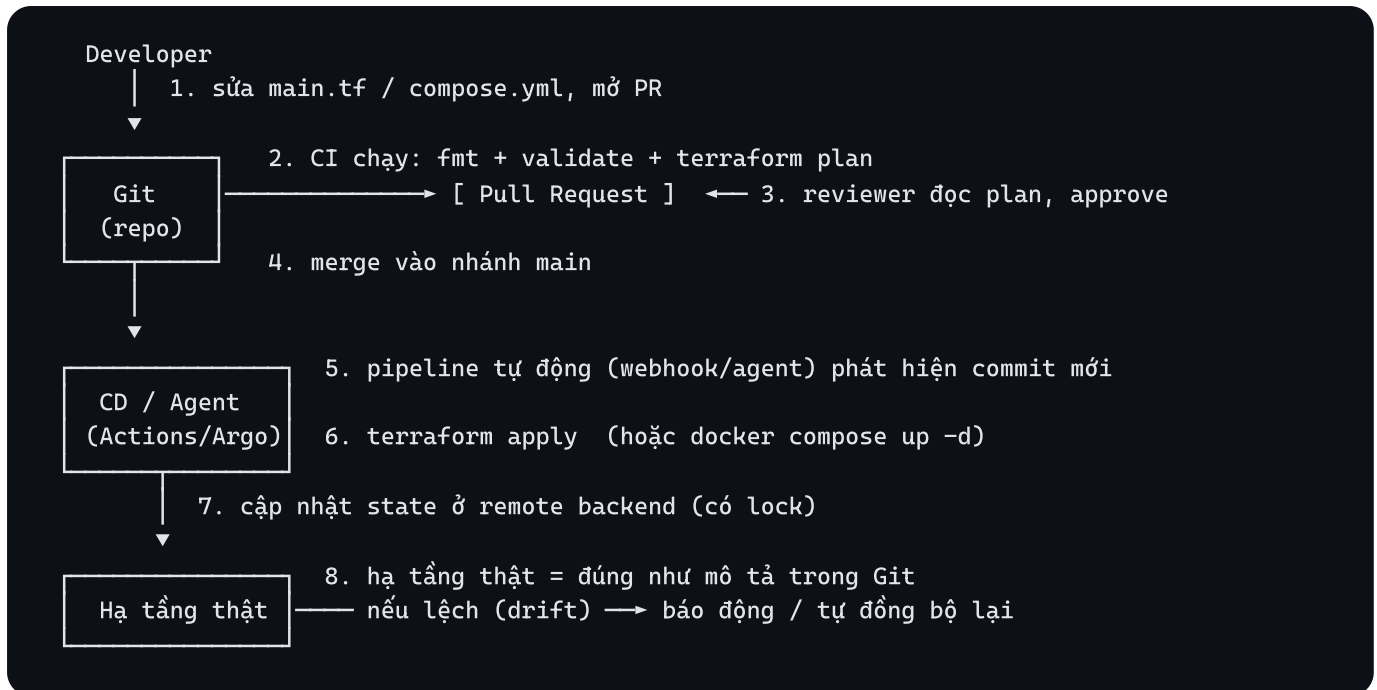
Lỗi thường gặp & cách xử lý

Lỗi thường gặp	Nguyên nhân	Cách xử lý
<code>Error: Cannot connect to the Docker daemon</code>	Docker Desktop chưa chạy hoặc sai <code>host</code> trong provider	Bật Docker Desktop; kiểm tra <code>host</code> (Windows npipe, Linux/Mac unix socket)
<code>port is already allocated</code>	Cổng 8080/3000 đã bị tiến trình khác chiếm	Đổi <code>external</code> /port map hoặc tắt tiến trình đang chiếm cổng
<code>terraform apply</code> đòi tạo lại resource đã có	State bị mất/lệch với thực tế (xóa <code>.tfstate</code>)	Dùng <code>terraform import</code> để nạp lại resource vào state, hoặc <code>destroy</code> sạch rồi apply
<code>app</code> crash <code>ECONNREFUSED</code> tới db	App start trước khi Postgres sẵn sàng	Thêm <code>healthcheck</code> cho db và <code>depends_on: condition: service_healthy</code>
Commit lỡ <code>terraform.tfstate</code> chứa secret	Chưa cấu hình <code>.gitignore</code>	Thêm vào <code>.gitignore</code> , xóa khỏi lịch sử (<code>git rm --cached</code>), xoay (rotate) secret bị lộ

Quy tắc vàng: mọi thứ có thể sinh lại thì đừng commit (`.terraform/`, state); mọi secret thì đừng bao giờ commit.

Mở rộng (Về nhà): phác thảo pipeline GitOps

GitOps = Git là *nguồn sự thật duy nhất*; mọi thay đổi config đi qua PR, và khi merge thì hệ thống **tự động apply** để hạ tầng thật luôn khớp với repo. Hãy phác lường:



Nhiệm vụ về nhà: viết một GitHub Actions workflow đơn giản chạy `terraform plan` trên mỗi PR và `terraform apply` khi merge vào `main`; ghi chú nơi lưu state (remote backend) và cách bảo vệ secret (GitHub Secrets). Trình bày vì sao tách plan (ở PR) và apply (ở main) giúp an toàn.

Lab 5

Bài giải Lab 5 — CLI tool DevOps bằng Go

Mục tiêu bài lab

Trong bài này chúng ta xây dựng một công cụ dòng lệnh (CLI) nhỏ nhưng thực chiến bằng Go: một **health-check tool** nhận vào danh sách URL/service, gọi HTTP song song, in trạng thái từng dịch vụ và **thoát với mã lỗi khác 0 nếu có service down**. Đây là dạng công cụ cực kỳ phổ biến trong DevOps vì nó có thể chèn thẳng vào một bước CI/CD để “chặn” pipeline khi hệ thống không khỏe.

Vì sao chọn Go? Go biên dịch ra **một binary tĩnh duy nhất**, không cần cài runtime trên máy đích — rất hợp để thả vào Jenkins agent hay container. Go cũng có concurrency (goroutine) gọn nhẹ, hợp với việc ping nhiều URL cùng lúc.

Kết thúc bài, bạn sẽ: khởi tạo module, viết CLI, build, cross-compile cho Linux/Windows, và nhúng binary vào Jenkins pipeline.

Phương án thay thế (nếu chưa quen HTTP): làm một tool đọc file **inventory** và in danh sách host theo môi trường (dev/staging/prod). Bài này ưu tiên ví dụ health-check vì nó minh họa rõ giá trị của exit code trong CI.

Bước 1 — Khởi tạo module (go mod init)

Tạo thư mục dự án và khởi tạo module. Module là đơn vị quản lý phụ thuộc của Go; lệnh **go mod init** sinh ra file **go.mod** khai báo tên module và phiên bản Go.

```
mkdir healthcheck && cd healthcheck
go mod init healthcheck
```

Vì sao cần bước này? Không có **go.mod**, Go coi thư mục là code rời rạc theo GOPATH cũ, khó quản lý dependency. Có module rồi, sau này nếu bạn thêm thư viện (ví dụ **cobra**), **go** sẽ tự ghi vào **go.mod** / **go.sum** để build tái lập được.

Bước 2 — Viết CLI health-check (flag, HTTP, goroutine)

Tạo file **main.go**. Đây là phần lõi — hãy đọc kỹ giải thích bên dưới.

```

package main

import (
    "context"
    "flag"
    "fmt"
    "net/http"
    "os"
    "strings"
    "sync"
    "time"
)

// result lưu kết quả check của một URL để in ra sau khi tất cả goroutine chạy xong.
type result struct {
    url      string
    status   string
    ok       bool
}

func checkURL(ctx context.Context, url string, timeout time.Duration) result {
    // Mỗi request có context với deadline riêng ⇒ không bị treo vô hạn.
    ctx, cancel := context.WithTimeout(ctx, timeout)
    defer cancel()

    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return result{url, "REQUEST ERROR: " + err.Error(), false}
    }

    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        // Lỗi mạng, DNS, timeout, connection refused... đều rơi vào đây.
        return result{url, "DOWN: " + err.Error(), false}
    }
    defer resp.Body.Close()

    // Quy ước: 2xx và 3xx coi là khỏe; còn lại là bất thường.
    ok := resp.StatusCode < 400
    label := "OK"
    if !ok {
        label = "BAD"
    }
    return result{url, fmt.Sprintf("%s (HTTP %d)", label, resp.StatusCode), ok}
}

func main() {
    // flag: khai báo tham số dòng lệnh -urls và -timeout.
    urlsFlag := flag.String("urls", "", "Danh sách URL, phân tách bằng dấu phẩy")
    timeoutFlag := flag.Duration("timeout", 5*time.Second, "Timeout mỗi request")
    flag.Parse()

    if *urlsFlag == "" {
        fmt.Fprintln(os.Stderr, "Thiếu -urls. Ví dụ: -urls=https://a.com,https://b.com")
        os.Exit(2) // exit code 2: lỗi sử dụng sai tham số.
    }
}

```

```

}

urls := strings.Split(*urlsFlag, ",")

var wg sync.WaitGroup
results := make([]result, len(urls))

// Gọi tất cả URL song song bằng goroutine.
for i, u := range urls {
    wg.Add(1)
    go func(idx int, url string) {
        defer wg.Done()
        results[idx] = checkURL(context.Background(), strings.TrimSpace(url),
*timeoutFlag)
    }(i, u)
}

wg.Wait() // Chặn cho đến khi mọi goroutine hoàn thành.

failed := 0
for _, r := range results {
    fmt.Printf("[%4s] %s → %s\n", statusMark(r.ok), r.url, r.status)
    if !r.ok {
        failed++
    }
}

fmt.Printf("\nTổng: %d, Down: %d\n", len(results), failed)

if failed > 0 {
    os.Exit(1) // Có service down ⇒ exit ≠ 0 để CI đánh dấu FAIL.
}
}

func statusMark(ok bool) string {
    if ok {
        return "PASS"
    }
    return "FAIL"
}
}

```

Giải thích các phần và vì sao:

- **flag** : chuẩn thư viện chuẩn để nhận tham số. Dùng nó thay vì tự parse `os.Args` giúp có `-h` tự động và ép kiểu (như `Duration`) miễn phí.
- **context.WithTimeout** : đặt deadline cho mỗi request. **Vì sao quan trọng?** Nếu một service treo, không có timeout thì goroutine (và cả pipeline CI) sẽ đứng mãi. Timeout biến “treo vô hạn” thành “lỗi rõ ràng”.
- **goroutine + sync.WaitGroup** : mỗi URL chạy trong một goroutine, `WaitGroup` đếm và `Wait()` cho đến khi xong hết. Nhờ vậy check 10 URL mất khoảng thời gian của URL chậm nhất, không phải tổng của tất cả.

- Ghi kết quả vào `results[idx]` theo **chỉ số cố định** nên không cần khóa (mỗi goroutine ghi ô riêng) — tránh race condition mà không cần mutex.
- `os.Exit(1)` **khi có down**: đây là mấu chốt cho CI. Jenkins/GitLab đọc exit code: 0 = pass, khác 0 = fail. Nhờ đó tool tự động “chặn” deploy khi hệ thống lỗi.

Bước 3 — go build & chạy thử

```
go build -o healthcheck
./healthcheck -urls=https://example.com,https://httpstat.us/500
```

`go build -o healthcheck` biên dịch ra binary tên `healthcheck` trong thư mục hiện tại. Chạy thử với một URL tốt và một URL trả 500 để thấy công cụ phân biệt được PASS/FAIL và in tổng kết. Kiểm tra exit code ngay sau khi chạy:

```
echo $? #1 nếu có service down, 0 nếu tất cả OK
```

Bước 4 — Cross-compile cho Linux/Windows

Sức mạnh lớn nhất của Go: đổi hai biến môi trường là ra binary cho HĐH/kiến trúc khác, **không cần máy đích**.

```
GOOS=linux GOARCH=amd64 go build -o healthcheck
GOOS=windows GOARCH=amd64 go build -o healthcheck.exe
```

`GOOS` chọn hệ điều hành, `GOARCH` chọn kiến trúc CPU. Vì sao hữu ích? Bạn có thể build ngay trên máy dev (macOS/Windows) ra binary Linux để chạy trên Jenkins agent hoặc container Linux — không cần Docker chỉ để biên dịch.

Lưu ý: trên PowerShell (Windows), cú pháp set biến khác Bash. Dùng `$env:GOOS="linux"; $env:GOARCH="amd64"; go build -o healthcheck` rồi nhớ reset lại sau.

Bước 5 — Đóng gói binary trong Jenkins pipeline

Ý tưởng: một stage build binary, một stage dùng chính binary đó để health-check sau khi deploy, và lưu binary làm artifact.

```

pipeline {
  agent any
  stages {
    stage('Build CLI') {
      steps {
        sh 'GOOS=linux GOARCH=amd64 go build -o healthcheck'
      }
    }
    stage('Deploy') {
      steps {
        sh './deploy.sh' // bước deploy ứng dụng của bạn
      }
    }
    stage('Health check') {
      steps {
        // Nếu healthcheck trả exit ≠ 0, stage này FAIL và pipeline dừng.
        sh './healthcheck -
urls=https://staging.myapp.com/health,https://staging.myapp.com/api/ping'
      }
    }
  }
  post {
    always {
      archiveArtifacts artifacts: 'healthcheck', fingerprint: true
    }
  }
}

```

Vì sao thiết kế thế này? Stage **Health check** chạy **sau** deploy, và vì tool trả exit khác 0 khi có service down, Jenkins tự đánh dấu build FAIL — biến health-check thành một “cổng chất lượng” (quality gate). **archiveArtifacts** lưu binary lại để tải về hoặc dùng cho stage sau, **fingerprint: true** giúp truy vết phiên bản binary giữa các build.

Kiểm chứng kết quả

- Chạy **./healthcheck -urls=<một_URL_tốt>** → in **PASS**, **echo \$?** trả **0**.
- Thêm một URL chắc chắn hỏng (ví dụ **http://localhost:9999**) → in **FAIL ... DOWN: connection refused**, exit code **1**.
- Chạy **file healthcheck** (Linux) để xác nhận là ELF 64-bit; **file healthcheck.exe** phải là PE32+ cho Windows.
- Trong Jenkins: mở build log, xác nhận stage **Health check** chuyển FAIL khi bạn cố tình trở vào URL down, và artifact **healthcheck** xuất hiện ở trang build.

Ứng dụng AI: generate boilerplate

AI rất hợp để sinh khung code lặp đi lặp lại, còn bạn tập trung vào logic nghiệp vụ.

Ví dụ prompt:

"Viết một CLI Go dùng package `flag` nhận cờ `-urls` (danh sách phân tách bằng dấu phẩy), gọi HTTP GET song song bằng goroutine + `sync.WaitGroup`, mỗi request có timeout qua `context`, in trạng thái từng URL và `os.Exit(1)` nếu có URL trả mã ≥ 400 hoặc lỗi mạng. Chỉ dùng thư viện chuẩn."

Checklist review code AI sinh ra (đừng tin tưởng mù quáng):

- ☐ **Xử lý error:** có kiểm tra `err` sau `http.Do` và sau `NewRequest` không? Có `defer resp.Body.Close()` không (rò rỉ kết nối nếu thiếu)?
- ☐ **Context/timeout:** mỗi request có deadline riêng không, hay dùng client mặc định treo vô hạn?
- ☐ **Exit code:** có `os.Exit` khác 0 khi lỗi không? (AI hay quên phần này — mà đây là điểm mấu chốt cho CI.)
- ☐ **Race condition:** nhiều goroutine ghi chung slice/map có an toàn không (ghi theo index riêng hay dùng mutex/channel)?
- ☐ **Tài nguyên:** có giới hạn số goroutine khi danh sách rất lớn không?

Lỗi thường gặp & cách xử lý

Lỗi thường gặp	Nguyên nhân	Cách xử lý
<code>go: cannot find main module</code>	Chưa chạy <code>go mod init</code> hoặc sai thư mục	Chạy <code>go mod init healthcheck</code> tại thư mục dự án
Chương trình treo, không thoát	Thiếu timeout, service đích không phản hồi	Dùng <code>context.WithTimeout</code> cho mỗi request như ở Bước 2
Exit code luôn = 0 dù có service down	Quên <code>os.Exit(1)</code> , chỉ <code>fmt.Println</code> lỗi	Đếm số <code>failed</code> và <code>os.Exit(1)</code> khi <code>failed > 0</code>
<code>fatal error: concurrent map writes</code>	Nhiều goroutine ghi chung map	Ghi vào slice theo index, hoặc dùng channel/mutex
Binary Linux không chạy trên agent	Build nhầm GOOS (ví dụ ra binary macOS)	Đặt <code>GOOS=linux GOARCH=amd64</code> khi cross-compile

Lưu ý: khi chạy trong container scratch/alpine, nếu gọi HTTPS mà báo lỗi certificate, hãy cài `ca-certificates` trong image hoặc build với `CGO_ENABLED=0` để có binary tĩnh hoàn toàn.

Mở rộng (Về nhà): thêm subcommand

Nhiệm vụ: biến tool một-lệnh thành tool nhiều **subcommand**. Gợi ý thêm:

- version**: in phiên bản build. Có thể nhúng version lúc build bằng `-ldflags "-X main.version=1.0.0"` để CI tự gắn số build.
- check-tcp**: thay vì HTTP GET, chỉ mở kết nối TCP bằng `net.DialTimeout("tcp", host:port, timeout)` để kiểm tra cổng (hợp với DB, Redis không nói HTTP).

Cách làm thủ công với thư viện chuẩn: dùng `flag.NewFlagSet` cho mỗi subcommand rồi rẽ nhánh theo `os.Args[1]`:

```
./healthcheck version
./healthcheck check-tcp -host=db.internal:5432
./healthcheck check -urls=https://a.com
```

Khi số subcommand tăng, nên chuyển sang thư viện **cobra** (chuẩn de-facto của các CLI DevOps như kubectl, docker). Cobra lo giúp bạn parsing, help tự động, và cấu trúc lệnh lồng nhau — code sạch hơn nhiều so với tự quản **flag.NewFlagSet** .

Hoàn thành phần này, bạn đã có một CLI DevOps đủ dùng thực tế và một quy trình build/đóng gói vào CI hoàn chỉnh.

PHẦN III

Tham khảo

Reference — Ebook & Video

Cách thêm link: mở file `src/content/reference/index.md`, thêm một dòng vào đúng mục theo mẫu `- [Tên tài liệu](https://link) - ghi chú ngắn`. Lưu lại là xong, không cần đụng code.

Tổng quan / DevOps

- [The Phoenix Project \(sách\)](#) — tiểu thuyết kinh điển về tư duy DevOps.
- [The DevOps Handbook \(sách\)](#) — thực hành CI/CD, flow, feedback.
- *Thêm link của bạn ở đây...*

Session 1 & 2 — Jenkins (CI/CD)

Ebook / Tài liệu

- [Jenkins User Documentation](#) — tài liệu chính thức.
- [Jenkins Pipeline Syntax](#) — tham chiếu Declarative pipeline.
- *Thêm ebook của bạn ở đây...*

Video

- *Dán link YouTube/khóa học của bạn ở đây...*

Session 3 — Ansible (Configuration Management)

Ebook / Tài liệu

- [Ansible Documentation](#) — tài liệu chính thức.
- [Ansible Best Practices](#) — mẹo tổ chức playbook/role.
- *Thêm ebook của bạn ở đây...*

Video

- *Dán link video của bạn ở đây...*

Session 4 — Config as Code (IaC)

Ebook / Tài liệu

- [Terraform Documentation](#) — tài liệu chính thức.
- [OpenTofu Docs](#) — bản mã nguồn mở của Terraform.
- [Docker Compose Docs](#) — mô tả stack bằng YAML.
- [Argo CD Docs](#) — GitOps trên Kubernetes.
- *Thêm ebook của bạn ở đây...*

Video

- *Dán link video của bạn ở đây...*

Session 5 — Golang cho DevOps tool

Ebook / Tài liệu

- [A Tour of Go](#) — học nhanh cú pháp Go.
- [Effective Go](#) — idiom và cách viết Go tốt.
- [Cobra \(CLI framework\)](#) — dựng CLI có subcommand.
- *Thêm ebook của bạn ở đây...*

Video

- *Dán link video của bạn ở đây...*